CS492 Senior Design Project II

Final Report

Elif Ercan 22201601

Bilge İdil Öziş 22102365

Kağan Rehber 22101777

Mennatallah Abouelenin 22101539

Bilgehan Tuğcu 22202667

1. Introduction

Sproutly is a cross-platform mobile productivity application designed to help users manage tasks, reduce digital distractions, and develop consistent focus habits through a supportive and gamified experience. The application targets students and everyday users who need help organizing their tasks, maintaining focus, and developing productive routines. Sproutly builds on existing productivity approaches by combining task management, customizable focus sessions, app restriction features, AI assistance, and virtual companion support in one integrated system. This combination allows the system to support both practical productivity needs and long-term habit formation.

The main user flow in Sproutly begins with planning a task and continues with starting a focus session, following the rules of a selected coaching mode, and receiving progress feedback at the end of the session. Coaching modes allow users to choose different levels of accountability, ranging from Casual mode with gentle reminders to Extreme mode with stricter cooldowns and stronger penalties. In this way, Sproutly aims to provide flexibility for different user preferences while still helping users develop sustainable productivity habits.

AI-assisted features are used to reduce cognitive load and improve personalization. These features include breaking down complex tasks into smaller steps, verifying proof of work submissions, and generating personalized focus recommendations based on user activity. By placing AI support around planning, reflection, and verification, Sproutly helps users make productivity decisions without replacing their control over the system.

This report documents the final design and implementation of Sproutly. It explains the final system architecture, implementation details, data management, test cases and their results, maintenance considerations, and reflections on engineering design decisions, teamwork, and knowledge gained throughout the project.

2. Requirements Details

2.1 Functional Requirements

User Registration and Profile: Registration and login are handled through email via Firebase Authentication [3]. When a new user signs up, they go through a name setup screen before they can access the rest of the app. From the profile screen, users can change their coaching mode, rename their companion, retake the personality quiz, view their completed task history, or delete their account. Moreover, A "Forgot Password" option on the login screen lets users request a reset link, which is then sent to their email. Signing out redirects the user to the login page and blocks backward navigation to the dashboard.

Personality Questionnaire: Right after signing up and entering their name, users are shown a personality quiz with 20 multiple-choice questions. The questions cover five behavioral axes: pressure tolerance, resilience, motivation, structure preference, and social drive. Each

of the four answer choices per question contributes weighted points to one or more of these axes. Once the quiz is done, the raw scores get normalized to a 0 to 100 scale and the system classifies the user to the closest archetype out of ten options by computing weighted Euclidean distance against predefined centroids. The archetypes are Anxious Perfectionist, Sensitive Introvert, Social Butterfly, Optimistic Planner, Competitive Achiever, Disciplined Grinder, Chill Procrastinator, Creative Chaotic, People Pleaser, and Steady Pragmatist. Each one maps to a companion personality (gentle, cheerleader, tough, sassy, or balanced), which shapes how the companion talks to the user throughout the app. The result screen shows the archetype name and description, and a radar chart of axis scores. Users who do not want to take the quiz can skip it, in which case they get the default "balanced" personality for their companion. Retaking the quiz later is possible from the profile screen.

Coaching Modes: There are four coaching modes controlling how strict the app is during focus sessions, how much XP the user earns, and how harsh the penalties are for quitting early. Casual mode gives 0.8x XP with no quit penalties and soft nudges. Standard mode gives 1.0x XP, deducts 10 XP on quit, tracks streaks, and requires a 45-second wait before the user can override a blocked app. Harsh mode gives 1.3x XP, deducts 25 XP and 1 level on quit, imposes a 2-day cooldown before the next session, requires a 120-second override wait, and forces the user to type "I QUIT" to confirm quitting. Extreme mode gives 2.0x XP but resets the companion to level 1 on quit, imposes a 5-day cooldown, requires a 300-second override wait, and makes the user type "I QUIT" and tap five times to confirm. Each mode also has separate configurations for Android and iOS. On Android, app blocking ranges from soft to aggressive depending on the mode. iOS cannot block apps in the background, so the penalties on the iOS side are scaled up to compensate. Switching between modes is allowed but comes with cooldown periods. For example, downgrading from Extreme to Harsh requires waiting 7 days. The user's selected mode is saved in SharedPreferences and loaded at the start of each session.

App Blocking: On Android, when a focus session is active, the app blocks whichever distracting apps the user has selected. If the user tries to open one of these blocked apps, a full-screen overlay pops up showing the companion and a "Return to Focus" button. How easily the user can override this block depends on the coaching mode. In Casual mode, overriding is basically free. In Standard and Harsh modes, overriding costs XP and may trigger a cooldown. In Extreme mode, the user gets exactly one override per session, and using it fails the entire session and resets the companion. Setting up app blocking requires the user to grant Usage Access and Display Over Other Apps permissions, with an additional Accessibility Service permission needed for Extreme mode. A dedicated onboarding screen walks users through each permission step by step. On iOS, true background app blocking is not possible due to OS restrictions, so the app relies on notifications and heavier penalties instead.

Focus Timer: Users pick between two timer types when starting a focus session. Pomodoro mode [7] counts down from a user-configured focus duration, then switches to a short break, and repeats. The defaults are 25 minutes of focus, 5-minute short break, and 15-minute long

break after every 4 focus intervals, but all of these values are customizable and saved for the next session. The transitions between focus and break happen automatically. Chronometer mode simply counts upward until the user decides to end the session manually. Both modes can be paused and resumed without losing any time. On Android, a foreground service keeps the timer running when the app goes to the background, and when the user comes back, the timer correctly reflects the elapsed time. Quitting mid-session shows a warning dialog specific to the coaching mode that explains the penalty before the user confirms. After the session ends, duration, coaching mode, and XP earned are passed to the session summary screen and saved in the analytics records.

Companion System: At first launch, the user picks one of six animated companions: Fox, Bunny, Owl, Hedgehog, Cat, or Penguin. Each companion is a Rive animation [4] driven by a state machine with three inputs: a sleeping toggle, an emotion input (idle, happy, or sad), and an accessory slot (none, bow, or hat). Companions earn XP from focus sessions. The amount of XP depends on session duration times the coaching mode multiplier. Leveling up follows the formula xpToNext = (100 * level * 1.5), and accessories unlock at specific levels (bow at level 2, hat at level 3). The companion's mood changes based on what the user does: it gets happy after a good session and sad after a failed one or a missed streak. In Extreme mode, quitting a session resets the companion all the way to level 1. The companion shows up on the home screen, the companion detail screen, the active timer screen, the session summary, the task screen, and the weekly review. Users can rename their companion from the profile screen. What the companion actually says depends on the personality assigned from the quiz, so a "tough" companion talks differently than a "gentle" one.

Task Management: Users create tasks by entering a title and an optional description. Once the user types a description, the app sends it to the AI backend, which infers the task type (study, work, or personal), the proof type (reading, written, digital, or general), and an importance level from 1 to 5. Deadlines are optional and can be set with a date and time picker. On the task list, incomplete tasks appear first, sorted by how close their deadline is and then by importance. There is also a task history screen that shows completed tasks grouped by the date they were finished. Tasks can be edited or deleted from the list.

AI Task Breakdown: When a task feels too large to tackle at once, the user can trigger a breakdown. The app sends the task title and description to the backend, which asks the Claude API to split it into smaller, ordered subtasks. Each subtask comes back with its own importance level. The user reviews the generated list and confirms before the subtasks are added. Users can choose which subtask to add or whether to add the task as a single task instead.

Proof of Work: After a focus session, the user can upload a photo of their completed work as proof. The proof upload screen lets them take a photo with the camera or pick one from the gallery. The image gets sent to the FastAPI backend [5] along with the task description, coaching mode, and proof type. The backend uses a two-call approach: the first call scores the proof objectively at a low temperature, and the second generates toned feedback at a higher temperature matching the coaching mode's strictness. The result screen shows a

confidence score from 0 to 100, an objective verdict, and itemized feedback explaining the decision. If the user thinks the AI got it wrong, they can file an appeal through the "Objection!" feature, where they write their argument and the AI reconsiders. The Proof of Work feature can be turned off from profile settings.

Mood Check-ins: The mood check-in dialog asks the user two things: how they are feeling (5 levels from Low to Amazing, shown as emojis) and their energy level (1 to 5). Based on this combination, a scoring algorithm picks the most fitting task from the user's active list. The scoring takes deadline urgency, task importance, and the user's current state into account. When energy is high, it favors important and harder tasks. When mood or energy is low, it leans toward easier ones. The recommended task is shown with a short explanation of why it was picked.

Monthly Calendar: The calendar shows a month view where tapping any day triggers a smooth animation: the other weeks collapse and the selected day's timeline expands below the calendar grid. The timeline displays three kinds of items, each with a colored dot: one-time tasks pulled from the task list, one-time events stored with specific dates and times, and recurring weekly activities. A red line marks the current time on the timeline. Items due within 24 hours show up with peach-colored dots, while normal items use green. A Rive companion peeks out between the calendar grid and the timeline, occasionally showing contextual messages. Recurring weekly activities live in SQLite [8] and have a day of week, start time, end time, description, and color. One-time events are stored in their own table with a date, time range, title, and description.

Notes: A notes system is accessible from the bottom navigation bar. The notes screen has a scratchpad for quick thoughts (which can be turned into notes or tasks with one tap), a topic filter bar, and a scrollable list of saved notes with pinned ones shown first. For structured note-taking, there are five built-in templates: Lecture Notes, Meeting Notes, Study Guide, Research Notes, and Brainstorm. Template notes use a section-based editor where each section has a colored header and its own rich text area powered by flutter_quill [11]. Headers are color-coded by section name and template type, and users can change them by long-pressing. Sections can be added, renamed, and deleted. Checklist sections like Action Items provide checkbox rows with strikethrough on completion. Free-form notes open as a plain rich text editor with a formatting toolbar, and can optionally be converted into section mode. Notes auto-save on back press only when content has changed. They can be pinned, deleted, and searched by title, content, or topic. The Brainstorm template opens a canvas where users create colored thought bubbles, drag them around, and connect them by long-pressing one and tapping another. AI features let users summarize a note into bullet points or find connections between a note and their existing tasks.

Weekly Progress Review: Once a week, the app puts together a review showing the user's daily focus time, the number of completed tasks compared to the planned task count, and the user’s streak status. The review screen includes AI-written feedback with suggestions based on the user's recent activity, the priority and deadline of upcoming tasks, the number of sessions quit and attempts to open blocked applications. The companion animation is shown

alongside the feedback. Review data is cached locally so the app does not re-fetch it every time the screen is opened.

XP, Levels, and Streaks: Progression is tracked through XP and levels tied to focus sessions. XP per session equals session duration times the coaching mode multiplier (0.8x for Casual, 1.0x for Standard, 1.3x for Harsh, 2.0x for Extreme). Sessions with zero focus time give zero XP no matter the mode. The XP needed to reach the next level is calculated as (100 * level * 1.5). Streaks count consecutive days of focus session activity. Breaking a streak has different consequences depending on the mode: Standard resets the streak counter, while Harsh and Extreme add cooldown days on top. The home screen shows weekly stats (focus minutes, completed tasks, streak count) in colored bubbles.

Achievements and Badges: An achievement system tracks milestones like finishing a certain number of tasks, keeping a streak going, or reaching specific companion levels. When an achievement unlocks, a popup with the companion animation notifies the user.

Password Reset: If a user forgets their password, they can request a reset from the login screen. Firebase Authentication sends a reset email to the address on file. The app checks the email format before sending and shows a success or error message afterward.

2.2 Non-Functional Requirements

2.2.1 Usability

- The application should offer an intuitive, clean, and easy-to-use interface that can be used daily.
- The user experience should remain consistent across iOS and Android platforms, ensuring that users can use the app easily on any device while respecting each platform's native patterns.
- All core features, like starting a focus session, adding tasks, or checking stats, should be within quick access and require a few steps.
- The interface should stay visually clear and pleasant, with readable fonts, suitable text sizes, and relaxing colors for concentration.
- The design should be smooth and reduce cognitive load by excluding unnecessary elements and presenting information in an organized, simple manner.

2.2.2 Reliability

- The app should function predictably and consistently, especially during focus sessions and task tracking.
- User progress, streaks, and statistics should remain accurate and consistent across all app activities.
- The app should remain stable without unexpected crashes and handle errors gracefully without interrupting core workflows.

2.2.3 Performance

- The app should run smoothly on standard iOS and Android devices without lag.
- Key features (focus timer, task creation, note-taking, mood check-ins) should load quickly to avoid disrupting user workflow.
- AI-based features such as weekly reports or task suggestions should return results within a reasonable time to maintain a good user experience.
- Animations (such as the companion or transitions) should feel responsive and not significantly impact performance.
- Background syncing operations should not slow down overall app usage.

2.2.4 Supportability

- The system should be easy to maintain, update, and extend over time without requiring major structural changes.
- Code should follow clean, modular architecture to allow quick identification, modification, and addition of features such as new coaching modes or analytics tools.
- The app should include clear documentation (well-documented APIs, code comments, developer guides) for core components (task management, focus sessions, AI features) to support future debugging and improvements.
- Error handling and logging mechanisms should enable efficient issue detection and troubleshooting.
- The design should allow smooth integration of future technologies or external APIs while keeping the existing system stable.

2.2.5 Scalability

- The system should be designed to handle a growing number of users without significant performance decline.
- The data structure should support increasing volumes of notes, tasks, schedules, and focus logs over time.
- The system should support multi-device access, ensuring user data remains consistent across phones and tablets as the number of devices and users grows.
- AI-based features should be designed so they can be improved or expanded in the future as the model evolves.
- The overall architecture should allow adding new coaching modes, companion features, or analytics tools without needing major redesigns.

3. Final Architecture and Design Details

3.1 Overview

Sproutly is a cross-platform mobile application that helps users stay focused during study or work sessions by tying consistency to a virtual companion. In a typical use case, the user plans or selects a task, starts a focus session, follows the rules of the selected coaching mode, and receives progress feedback based on the session outcome. Proof of work is an optional feature that can be enabled to support this feedback process. When enabled, the user can upload a photo at the end of a session, and the system sends it for AI-based review to verify whether it matches the completed work.

At a high level, the system follows a layered, modular structure. The Flutter [1] mobile client handles the user experience and the focus session flow. It owns all of the UI rendering, local session state, timer logic, and platform-specific behavior such as app restriction enforcement and widget rendering. The client does not access the database or call AI services directly; every data request and every AI-dependent action goes through the backend over RESTful API calls via HTTPS. A backend layer built with FastAPI [5] acts as the bridge between the app, the database, and AI components. The backend provides RESTful endpoints based on domain (user-facing endpoints such as create an account, modify an account, create/retrieve tasks, modify/manage schedule, create/search for notes, record session logs/the AI-facing endpoints, request task breakdowns, proof of work verification, and generating weekly reports). When the client sends a request that involves AI processing, the backend constructs the appropriate prompt or payload, forwards it to the external AI service, receives the response, validates and restructures it into the application's own data format, and returns the result to the client. The client never talks to AI services on its own, the backend owns that boundary entirely. The schema covers users, devices, personality profiles, tasks and subtasks, focus sessions, companions, schedules and schedule events, notes, distraction notes, weekly reports, proof of work records, and blocked app configurations. The backend is the only layer with direct read and write access to the database, and all queries and mutations pass through FastAPI endpoint handlers that enforce validation, authorization, and data consistency before any database operation executes. The system uses Anthropic's Claude Sonnet 4 [6] as the external AI service for AI-dependent features, including task breakdown into subtasks, proof of work photo verification, note summarization, and task recommendation. AI responses are not passed through to the client raw, the backend parses and restructures them before returning anything to the mobile app.

Inside the mobile application, features are organized around clear responsibilities to support modularity and maintainability. Core areas include task scheduling, configurable focus timers, coaching modes that define session strictness, app restriction features, and gamification features such as XP, streak tracking, and achievements. Coaching modes range from Casual to Extreme, with differences in how a user can override restrictions and how severe the penalties are when they break session rules. The intent is that users who want

gentle support can get nudges, while users who want stronger accountability can choose modes with stricter rules and stronger penalties.

The app restriction feature is designed to support focus through reminders and restrictions instead of only using strict app blocking. When the user attempts to open a blocked application during an active focus session, Sproutly displays a blocking screen that reminds the user to stay in the session. The user can either return to the focus session or use an emergency override if access to the blocked app is necessary. If the override option is used, the system applies consequences according to the selected coaching mode. The implementation also reflects the differences between mobile operating systems. On Android, stronger restriction behavior can be supported through usage access and overlay permissions, while iOS relies on reminders, notifications, and penalties because full background app blocking is limited by the operating system. The coaching mode selected by the user determines the severity of override penalties and the strength of restriction enforcement, and this configuration is stored so that it stays consistent.

Proof of Work and AI support are handled through the backend layer. After a session, a user can submit a photo as evidence of completed work. The client uploads the media file to the backend, which holds it in temporary storage, forwards it to claude-sonnet-4-20250514 for verification against the session's task description, and discards the media after the verification result is recorded. The backend then writes the updated progression data back to the database, and the client receives the final result and renders the appropriate feedback screen. Separating AI processing from the backend keeps the application lightweight and allows the verification logic to be updated without making changes to the mobile application. Finally, authentication and identity management are handled through Firebase Authentication [3].

3.2 Subsystem Decomposition

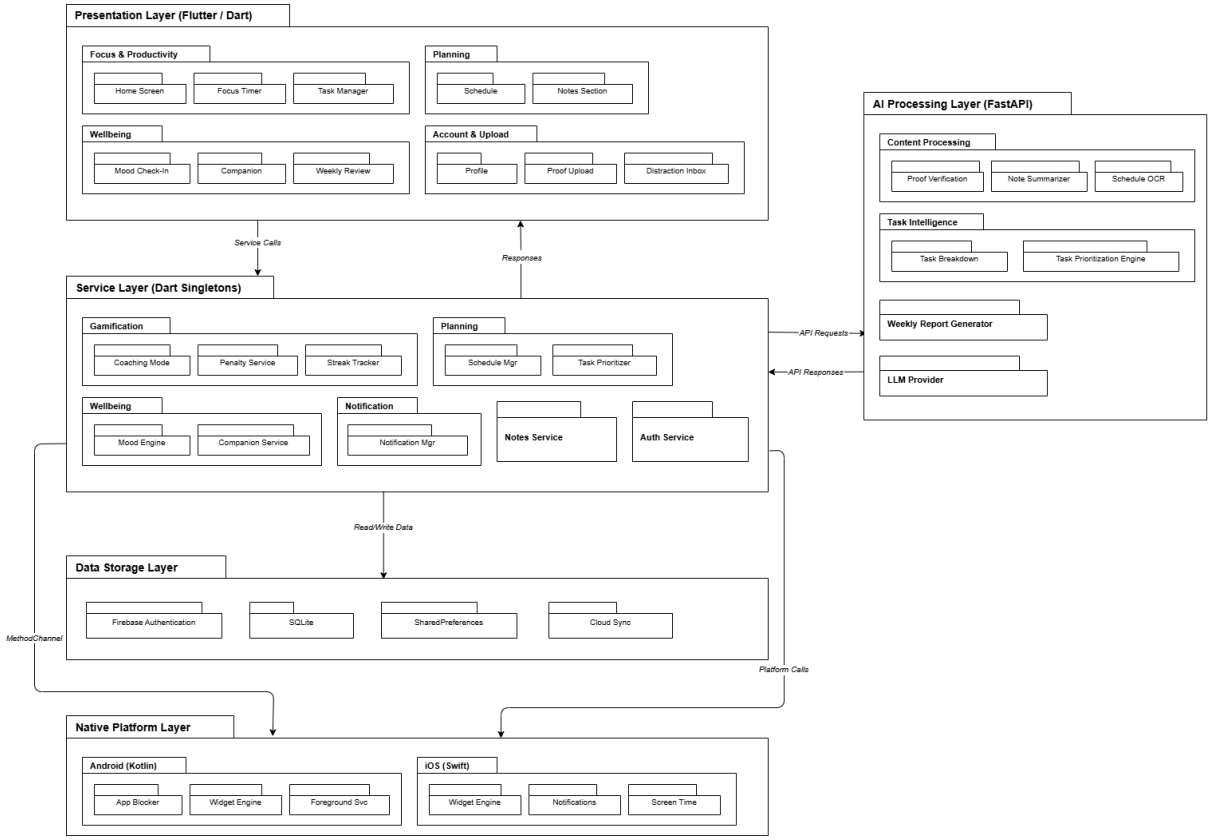

Figure 1: Subsystem decomposition diagram for Sproutly

3.2.1 Presentation Layer

Purpose: Provides the mobile user interface of Sproutly using Flutter. Enables users to route between screens, presents feedback and companion interactions and gets input from the user. The presentation layer is linked to backend and Firebase services.

Subsystems:

1. **Focus & Productivity:** Handles the main productivity flow including the dashboard screen which enables access to core modules, focus timer and task management.
2. **Planning:** Provides calendar based scheduling and note taking features to help users plan tasks and routines.
3. **Wellbeing:** Ensures the users' wellness with mood check-ins, companion features and weekly review. The companion module includes the screen to customize the companion as well.
4. **Account & Upload:** Handles account and profile related screens, proof upload for the Proof of Work feature and distraction inbox.

3.2.2 Service Layer

Purpose: Provide the necessary utility for certain features as singletons. These features include gamification elements, planner features, wellbeing related modules; notification, notes and authentication services.

1. **Gamification:** Coaching mode serves as a difficulty setting for the user. Penalty functionality creates negative incentives for certain coaching modes, scaling with the difficulty of the coaching mode selected. Streak tracking is used to track the user's consistency in using the app, to encourage them to follow their schedules, habits, and plans.
2. **Planning:** Planning functionality includes one-time, non-recurring activities (such as deadlines for tasks or projects) as well as weekly schedules laid out in the calendar by the user.
3. **Wellbeing:** This functionality manages information provided by the user about their habits such as mood and energy level.
4. **Notifications:** Notifications services are used to provide notifications under certain app usage conditions. Notifications may be sent if the user is distracted when the app is running in the background, or if a timer has been paused for too long during an active focus session. The frequency and content of notifications may differ based on the selected coaching mode.
5. **Notes Service:** Allows the user to take notes within the app. The user can choose from two forms of notes: one is the template form and the other is free form. The available templates are Lecture Notes, Meeting Agenda, Study Guide, Research Notes, and Brainstorm. The first four work in a similar way, with initial sections that users can fill in or modify however they want. They can also add new sections or remove existing ones. The app provides flexibility through a toolbar on each note editing page, where users can make text bold, add headers, insert bullet points, and so on. The Brainstorm template is different from the rest: instead of structured sections, it provides an empty canvas where users can add bubbles and draw connections between related ones. Beyond editing, the notes service also provides AI features. AI summarization pulls out key points from a note and highlights them with badges, and a connection-finding feature identifies related notes and tasks. Users can also upload a document or image and have the AI convert it into a note in the app's format.
6. **Auth Service:** Provides authentication services for user accounts. The service is handled on the Flutter client using Firebase Authentication. After authentication, newly created users can proceed and finalize their account setup, and existing users can sign in and out of the application.

3.2.3 AI Processing Layer

Purpose: Provides AI based functionality, including verifying proof uploads, summarizing notes, task breakdowns, and generating weekly reports.
Subsystems:

1. **Content Processing:** Manages content that needs AI confirmation such as verifying proof uploaded by the user, summarizing user notes.
2. **Task Intelligence:** Supports task planning features such as task breakdown which divides tasks into smaller parts and task suggestion which highlights more urgent or higher priority tasks for the user.
3. **Weekly Report Generator:** Generates weekly reports including the user's total focus time, number of focus sessions, number of completed tasks, streak information. The generated report also includes reminders and suggestions from LLM.
4. **LLM Provider:** Manages requests to the Claude LLM that Sproutly uses for confirming the proof sent by the user for the Proof of Work part. This module enables communication with Claude LLM and returns its outputs to the service layer.

3.2.4 Data Storage Layer

Purpose: Provides storage for user option preferences, user account information and any other relevant data, such as tasks and schedules, which the user may be utilizing in the application. Interfacing with the data storage layer is done through the backend.
Subsystems:

1. **SQLite, SharedPreferences (Android) / UserDefaults (iOS):** Provides on-device, local storage for data. This data may include user settings, user preferences, and more. Certain data may be cached in local storage.
2. **Firebase:** Cloud storage primarily used for account and login details.

3.2.5 Native Platform Layer

Purpose: Since certain features of the app can be implemented only on one specific OS, our app makes this distinction through the native platform layer. Each platform consists of its own native modules that are written in its own native language; in our case, this is Kotlin for Android and Swift for iOS. This layer is used to get access to these platform-specific features that can not be achieved with Flutter only. The native modules that each platform provides communicate with Flutter through a messaging system called the MethodChannel, which invokes native platform methods and receives the results.
Subsystems:

1. **Android (Kotlin):** This subsystem contains the following components: App Blocker, which blocks distracting apps from the user with some overriding and penalty conditions depending on the coaching mode during a focus session; Widget Engine responsible for rendering widgets in the home screen of the users phone displaying streak counts etc.; and Foreground Service, which maintains a persistent timer which will continue running even when the application is running in the background.
2. **iOS (Swift):** This subsystem contains the following components: Widget Engine, which behaves similarly to its Android counterpart, rendering home screen and lock screen widgets using the WidgetKit framework, displaying the companion, streak, level, and a live focus timer that updates every second during active sessions across three display sizes (small, medium, and lock screen); and Notifications Module,

responsible for managing local notification scheduling, which handles session-driven notifications that fire based on user behavior during focus sessions.

Layer-to-Layer Communication

1. **Presentation Layer → Service Layer:** The presentation layer makes service calls to the service layer. These service calls are made to do the following operations: task creation, starting/stopping focus sessions, mood check-ins, companion messages, authentication, and schedule management.
2. **Service Layer → Presentation Layer:** From the service layer to the presentation layer, responses of these service calls are made. Some examples can be given as follows: updated state data, streak information, coaching feedback, and companion responses.
3. **Service Layer → AI Processing Layer:** The service layer makes API requests to the AI processing layer. The API requests include: proof images for verification, notes for summarization, and task lists for prioritization.
4. **AI Processing Layer → Service Layer:** The AI processing layer responds to the service layer with API responses including summarized content for the notes, and orderings of prioritized tasks.
5. **Service Layer → Data Storage Layer:** Service layer reads from and writes to data storage layer. Some examples can be given as retrieving tasks, notes, and schedules from SQLite; storing and reading user preferences, streak counts, and the coaching mode from SharedPreferences; managing authentication through firebase.
6. **Service Layer → Native Platform Layer:** Two types of communication happen between these two layers. The first one, MethodChannel, is used to invoke the native Android functionality such as app blocking, widget updates, and foreground service management. The second one, Platform Calls, invokes native iOS functionality like rendering widgets, scheduling notifications, as well as integrating Apple's Screen Time API.

Advantages of the Architecture

1. **Layered Separation of Concerns:** The system has a layered design where each layer handles its own set of responsibilities. For example, there is a presentation layer, a service layer, a data storage layer, a native platform layer, and an AI processing layer. This improves maintainability and reduces coupling between components.
2. **Modularity:** The application is divided into modules, which simplify development, testing, and future maintenance. For example, there is a planning module, a gamification module, a wellbeing module, and a notification module.
3. **Cross-Platform Support:** The application is developed using Flutter, allowing it to support both Android and iOS. It also has access to platform-specific capabilities through its native platform layer, allowing it to use widgets, notifications, and app restriction capabilities provided by the native platforms.

4. **Centralized AI Processing:** All AI-related operations of the system are handled by the FastAPI backend. There is no AI-related operation handled directly by the mobile client. This improves security and makes it easier to integrate AI into the system, and also makes it easier to update it.
5. **Scalability and Extensibility:** Both AI-related components and backend services operate independently of each other. Hence, they can be scaled independently. This makes it easier to add new features to the system or add AI-related services to the system in the future.
6. **Security:** Authentication is done using Firebase Authentication, and the backend is the only layer that is allowed direct access to the database. This is to ensure that all operations are passed through validation and authorization checks.

3.3 Hardware/Software Mapping

The project is not built with, and does not depend on, any specific hardware components; therefore, this section is left blank.

3.4 Persistent Data Management

Sproutly uses local to manage persistent data. On the mobile device, SharedPreferences is used to store small, local application data such as session information, and user settings and preferences; allowing the app to maintain quick access to frequently used data. SQLite is used to store and access data which may appear in similar forms in large amounts, such as tasks and activities, which accumulate over time as app usage increases.

User authentication and identity management are handled through Firebase Authentication, providing a secure cloud-based sign-in system. While the core user data remains local to the device, the app communicates with a FastAPI backend to perform complex AI-driven operations, such as task breakdown, note summarization, and Proof of Work verification. This architecture allows core data and interactions to remain locally accessible, while external AI services are accessed through the backend to support advanced productivity features.

3.5 Access Control and Security

Security and privacy are treated as baseline requirements rather than add-ons. The system includes Firebase for authentication and identity management. The authentication system supports email-based registration and login as part of the core requirements. Firebase Authentication is used as the managed identity layer for the application, reducing the need to implement and maintain a custom authentication system. Firebase issues authentication tokens that the mobile client includes in every API request to the backend, and the backend validates these tokens on each incoming request before processing it, ensuring that no unauthenticated or unauthorized access reaches the database or AI services. Firebase also provides the flexibility to enforce session tokens and access control across the mobile client

and backend. Beyond that, the design aims to use encryption wherever possible, including encryption in transit for all client-backend communication and encryption at rest for sensitive user data in the database, and explicitly takes care of compliance goals set by privacy regulations such as KVKK [21], HIPAA [23], and GDPR [22].

4. Development/Implementation Details

4.1 Frontend Implementation

4.1.1 Flutter/Dart

The Sproutly mobile client is built entirely with Flutter [1] using Dart [2], enabling a single shared codebase to target both Android and iOS while still allowing platform-specific behavior where needed. Flutter's widget-based rendering model suits the project well as the application required a condensed and visually rich interface with many interactive components across multiple screens such as the focus timer, calendar, companion view, task management, notes, and mood check-in flows.

The application's screen structure is organized around a MainShell stateful widget that manages a BottomAppBar with a CircularNotchedRectangle cutout and a docked FloatingActionButton at the center. The five primary areas are: Home (companion overview and weekly summary), Calendar (monthly activities and one-time tasks), Focus (accessed via the central FAB, leading to timer setup and the active session flow), Tasks (task list management), and Notes (user notes). Navigation between these areas is handled by updating _currentIndex via setState, pushing the corresponding screen into a body switcher.

Authentication state is managed by an AuthWrapper widget that listens to FirebaseAuth.instance.authStateChanges() and gates entry into the main shell. After a successful sign-in, a _CompanionGate check determines whether the user has selected a companion yet; if not, they are pushed to CompanionSelectionScreen before reaching the main navigation.

Shared application state such as companion XP, level, mood, and coaching mode is stored in SharedPreferences via singleton service classes. Screens read from and write to these services directly and trigger local setState calls or reload data in initState/didChangeDependencies when returning from sub-screens. Persistent relational data (tasks and weekly recurring activities) is stored in a SQLite database [8] named sproutly_master_db.db, accessed through a DatabaseService singleton that wraps the sqflite [9] package.

All communication with the FastAPI backend [5] is handled via the standard http package. The primary endpoint for Proof of Work feature is POST /pow/verify, which accepts a multipart form upload containing the task description, coaching mode, proof type, and one or

more image files, and returns a structured JSON response with a verification verdict, confidence score, and itemized feedback. A separate POST /pow/appeal endpoint accepts a JSON body with the original result and the user's written appeal. Both requests are issued by ProofOfWorkService with a two-minute timeout. JSON response parsing is done using jsonDecode from dart:convert, with dedicated model classes (VerificationResult, VerificationScores, VerificationFeedback) that carry fromJson factory constructors. Firebase Auth [3] is used for user identity but authentication tokens are not currently forwarded to the backend API.

The focus session timer is implemented inside TimerActiveScreen as a stateful widget. A Dart Timer object fires on a one-second interval, decrementing _secondsRemaining in Pomodoro [7] countdown mode or incrementing elapsed time in chronometer mode. The timer supports two modes: _TimerMode.pomodoro and _TimerMode.chronometer, with Pomodoro intervals configured at setup (default: 25-minute focus, 5-minute short break, 15-minute long break, long break every four intervals). Phase transitions are triggered automatically via _beginPomodoroPhase() when the countdown reaches zero. Timer state exists only within the active screen's lifecycle; there is no background persistence of the timer itself. Session completion data is passed forward to the VerificationResultScreen and written to AnalyticsService using SharedPreferences with date-keyed entries.

The calendar module renders weekly recurring activities and one-time tasks using a custom-built scrollable grid widget in WeeklyCalendarScreen. Conflict detection and touch handling are computed client-side. Recurring weekly activities are fetched from SQLite via DatabaseService.getWeeklyActivities(), while one-time tasks are sourced from the TasksData list (an in-memory cache backed by SQLite). Task creation forms use manual imperative validation checks (isEmpty guards and SnackBar error messages) to check for empty names, missing time selections, and invalid durations before any write operation is triggered.

Image capture for proof-of-work submission is handled through the image_picker package [24], and file paths are resolved using path_provider [25]. SVG assets (including companion accessories rendered as overlays) are displayed using the flutter_svg [26] package.

4.1.2 Rive

Companion animations are implemented using Rive [4], with .riv files rendered inside Flutter via the rive package. Six companions are available as of now: Fox, Bunny, Owl, Hedgehog, Cat, and Penguin, all with an accessory overlay system to support user customization.

Each companion's .riv file contains a state machine with three named inputs: a sleeping boolean (SMIBool) that triggers the sleep animation, an emotion number (SMINumber) that maps to idle (0), happy (1), or sad (2), and an accessory number (SMINumber) that selects which accessory to display (0 = none, 1 = bow, 2 = hat). The Flutter application writes to these inputs at runtime by obtaining a StateMachineController via

StateMachineController.fromArtboard() inside a RiveAnimation.asset() widget's onInit callback, then calling findInput<T>() for each named input and setting .value directly.

Companion mood that is stored as a string ('idle', 'happy', 'sad', 'sleeping') in SharedPreferences under the companion_mood key, is mapped to the numeric emotion input when the screen initializes or when the companion state changes after a focus session completes. XP is calculated from focus session duration and a per-coaching-mode multiplier via calculateXpFromFocusTime(), and level thresholds follow the formula xpToNext = (100 * level * 1.5).round(). Accessories are unlocked at specific level thresholds (level 2 for bow, level 3 for hat) and stored as a list in SharedPreferences under unlocked_accessories. The companion is displayed on the home screen dashboard, the dedicated companion screen, and the timer active screen. The companion_selection_screen.dart and accessory_selection_screen.dart screens handle initial companion choice and accessory equipping respectively.

4.2 Storage Solutions

4.2.1 SharedPreferences | UserDefaults

SharedPreferences (Android) [10] and UserDefaults (iOS, planned) are used to store local data that is not critical for basic user functionality. When the user creates a focus session and enters values for timer fields ‘longBreak’, ‘shortBreak’ and ‘focus’ inside the ‘timer_setup_screen’ page; the values they start the focus session with are saved in UserPreferences for reuse. When the user finishes their focus session and re-enters the 'timer_setup_screen’ page later, they will see the values they last entered. Certain companion data, such as accessories, are also stored under SharedPreferences.

4.2.2 Firebase Database

A remote Firebase [3] database is used to handle core user account information, including login details and passwords. Account creation and deletion requests are made to, and handled by, the remote server.

4.2.3 SQLite Database

The SQLite database, managed by a versioned DatabaseService singleton, serves 4 distinct tables and stores the core data relevant to features found within the app. The tasks table handles one-time To-Dos; tracking completion and proof of work verification, importance levels, and deadlines. The weekly_activities table manages recurring weekly routines found within the calendar module. one_time_events table stores specific appointments that don’t repeat, unlike weekly_activities. The notes table provides storage that supports persistent scratchpads, pinned entries, and structured templates for organized record-keeping. The weekly summary uses data stored in these tables to generate a weekly overview for the user.

4.3 Backend

4.3.1 Python

4.3.1.1 General Setup

The backend is built with FastAPI [5]. The entry point is a single main.py that creates the FastAPI app, enables CORS for all origins, and mounts five routers, each owning a separate domain of the API: proof_of_work, weekly_review, companion, tasks, and notes. All AI-facing communication goes through the Anthropic Python SDK. The model name and API key are both loaded from environment variables, and the key is checked at the start of every request that needs AI. If the key is missing, the endpoint either returns a safe fallback response or raises a 500 error, depending on how critical the AI result is for that feature.

The companion router is currently a stub that returns static dummy data (level, XP, streak, mood, accessory). Companion state is managed entirely on the client side through SharedPreferences, so this endpoint exists as a placeholder for future server-side synchronization.

4.3.1.2 Proof of Work Verification

The proof_of_work router exposes two endpoints: POST /pow/verify for initial verification and POST /pow/appeal for the objection system. The verify endpoint accepts a multipart form upload with the task description, coaching mode, proof type, and up to five image files (each capped at 5 MB). Files go through a processing pipeline: extension and content-type validation, chunked reading to enforce size limits, image resizing via Pillow [13] (capped at 1024px on the longest side), format conversion (GIFs become PNGs, everything else becomes JPEG at 85% quality), and finally base64 encoding for the API payload. The verification logic itself is covered in Section 4.3.2.

The appeal endpoint receives the original scores, the user's written explanation, and the coaching mode. It does not re-process images. The model reconsiders the verdict based on the explanation and returns a new confidence value and a verdict summary.

4.3.1.3 Task Intelligence

The tasks router has two endpoints. POST /tasks/infer takes a task title and description and returns an inferred task type (Study, Office, or Personal), proof type, and importance level from 1 to 5. The response is validated and clamped so that types and proof categories always fall within the allowed set, and priorities stay between 1 and 5. POST /tasks/breakdown splits a task into 3 to 6 subtasks, each with a title, proof type, estimated duration in minutes, and a priority from 1 to 5. Responses are validated (minutes clamped between 5 and 120, max 6 subtasks returned).

4.3.1.4 Notes AI

The notes router provides POST /notes/summarize and POST /notes/connections. Summarization sends the note's plain text to Claude and returns bullet points. Connection finding sends the note content along with a list of the user's task titles and returns a JSON array of task-reason pairs identifying which tasks relate to the note and why.

4.3.1.5 Weekly Review

The weekly_review router generates personalized weekly feedback. The Flutter client collects the week's data locally (focus minutes per day, completed sessions, completed tasks, streak, blocked app attempts, override attempts, quit count, and upcoming tasks with deadlines), then sends it as a JSON payload to POST /weekly-review/generate. If the user has fewer than five completed sessions that week, the endpoint skips the AI call entirely and returns a handwritten fallback response built from the raw data, which saves API costs for users who barely used the app. If the AI call fails for any reason, the fallback response is returned instead, so the user always sees something. If the user has not started a session or completed any tasks yet, a message is displayed informing them that feedback and suggestions will become available after they complete more tasks and focus sessions.

4.3.1.6 Error Handling and Validation

JSON extraction from model responses is handled by a shared extract_json function that tries four strategies in order: direct parse, markdown code block extraction, brace-matching to find the first { and last }, and trailing-comma cleanup. LLM outputs are not always valid JSON on the first try, so the fallback chain matters. All API calls go through a retry wrapper that catches timeouts, 5xx errors, and connection failures, waiting with exponential backoff (2^attempt seconds) for up to two retries.

Every response is validated through Pydantic models [12] before being returned to the client. Scores are clamped to 0-100, confidence to 0.0-1.0, and string fields are truncated to prevent oversized payloads. If a field is missing or malformed, it falls back to a safe default instead of crashing the request.

Structured logging is set up through Python's standard logging module. Every request gets a short UUID prefix so that related log lines can be traced together. The logs capture file processing results, LLM call attempts and retries, JSON extraction failures, and final verdicts. This has been useful for debugging prompt issues during development, since the raw model responses are logged at debug level.

4.3.2 AI

4.3.2.1 Model and SDK

All AI features run through Anthropic's Claude API [6] using the claude-sonnet-4-20250514 model. The backend is the only layer that talks to the API; the mobile client never makes

direct AI calls. Each router constructs its own prompts, sends them through the Anthropic Python SDK, and parses the structured JSON response before returning anything to the client. Every prompt explicitly instructs the model to respond with JSON only, no markdown, no preamble. This constraint is what makes the extract_json fallback chain (described in 4.3.1.6) necessary, since the model does not always follow the instruction perfectly.

4.3.2.2 Proof of Work: Two-Call Design

The Proof of Work verification uses a two-call design to separate objective scoring from subjective feedback. The first call sends the uploaded images alongside a scoring-only prompt at temperature 0.5. It asks the model to return scores for completeness, quality, and effort (each 0 to 100) plus an overall confidence value, without any tone or coaching influence. Keeping the temperature low and the prompt neutral means that the same proof gets roughly the same numerical score regardless of which coaching mode the user picked.

The second call takes those scores and feeds them into a feedback prompt that includes the coaching mode's tone instructions. Casual mode gets a "supportive, friendly study buddy" tone, Standard gets "fair, constructive teacher," Harsh gets "demanding professor," and Extreme gets "elite mentor with the highest standard." This call uses extended thinking with a 2048-token thinking budget, giving the model space to reason about the student's specific work before writing the feedback. The result is that feedback references actual details from the submission rather than generic advice.

The final verdict (verified, needs_review, or rejected) is determined by comparing the confidence against per-mode thresholds. Casual verifies at 60% confidence, Standard at 75%, Harsh at 85%, and Extreme at 92%. This means the same proof could pass in Casual but fail in Extreme, which matches the design intent of each coaching mode.

4.3.2.3 Proof Type Context

Each proof type (reading, written, digital, general) injects a context block into the scoring prompt that tells the model what counts as valid evidence and what to ignore. For a reading task, the model is told not to penalize the lack of written notes or summaries, since reading is consumption, not production. For a digital task, it knows not to check whether code compiles or runs correctly. For a written task, it focuses on content substance and amount of writing rather than spelling or formatting. The general type tells the model to infer what "done" looks like from the task description itself. This prevents the common problem of the AI applying the wrong rubric to a submission.

4.3.2.4 Appeal Evaluation

When a user files an objection, the AI re-evaluates the verdict based on the original scores and the user's written explanation. The strictness of this re-evaluation depends on the coaching mode. In Casual mode, the stance is lenient: if the explanation is reasonable and not obviously made up, the verdict gets upgraded. In Standard mode, the stance is fair: the

explanation needs to point out a genuine gap in the original assessment. In Harsh mode, the stance is skeptical: only concrete evidence that the AI missed something gets accepted. In Extreme mode, the stance is near-immovable: only a clear factual error by the AI warrants a change, and even then the verdict may only go up to needs_review rather than straight to verified.

4.3.2.5 Task Intelligence Prompts

The task inference prompt asks the model to classify a task based on its title and description. The model picks a task type, a proof type, and a priority from 1 to 5. The prompt gives concrete anchors for priority: "final exam" or "due tomorrow" maps to high priority, "organize" or "quick" maps to low. The max token limit is set to 200 to keep the response fast since this call happens while the user is still on the task creation screen.

The task breakdown prompt asks for 3 to 6 subtasks, each with an estimated duration and priority. The priority rubric is spelled out in the prompt: 5 means the subtask blocks everything else, 4 is core content, 3 is standard, 2 is helpful but not critical, and 1 is optional polish. Subtasks that would hurt equally if skipped get equal priority. This level of detail in the prompt was added after early testing showed the model assigning 3 to everything without guidance.

4.3.2.6 Weekly Review Generation

For users with five or more sessions in the week, the weekly review prompt sends all collected stats and asks for one feedback paragraph and three suggestions. The prompt includes specific tone rules: no report-like phrases such as "the data indicates" or "maintaining momentum," and the feedback should sound like a thoughtful coach talking directly to the user. Structural rules require that at least two of the three suggestions reference specific upcoming tasks if task data is available, so the suggestions stay concrete rather than generic. If blocked app attempts are greater than zero, the prompt steers the third suggestion toward distraction management rather than wasting it on a general tip.

4.3.2.7 Notes AI

Note summarization uses a simple prompt asking for concise bullet points, capped at 10, focusing on key facts, definitions, and action items. The connection-finding prompt receives the note content and the full list of the user's task titles, and asks the model to identify related tasks with a one-sentence explanation per connection. Tasks that have no relation to the note are left out. Both prompts are straightforward single-call designs without the multi-step approach used in proof-of-work, since the stakes of getting a summary slightly wrong are much lower than misjudging a proof verdict.

4.3.3 Kotlin

4.3.3.1 Platform Channel Architecture

The Android-specific features are written in Kotlin and communicate with the Flutter layer through MethodChannels and EventChannels. MainActivity registers two plugins at startup: AppBlockerPlugin for app blocking during focus sessions, and WidgetPlugin for home screen widgets and the persistent focus notification. Both plugins implement FlutterPlugin and MethodChannel.MethodCallHandler, following the same pattern used by Flutter's own platform plugins. This means all native Android functionality is callable from Dart through simple string-based method invocations, and data passes back and forth as Maps and primitive types.

The app blocker also uses an EventChannel for real-time events. When the user opens a blocked app during a session, the native side pushes an event to the Dart side through a shared EventSink. This was necessary because MethodChannels are request-response only, and app blocking needs to notify the Flutter UI about violations as they happen, without the Dart side having to poll for them.

4.3.3.2 App Blocking System

App blocking is split across two classes: AppBlockerPlugin handles the Flutter communication and permission management, and AppBlockerService runs as a foreground service that does the actual monitoring.

The plugin exposes methods for checking and requesting the Android permissions that the blocking system needs: Usage Stats access (to see which app is in the foreground) and Overlay permission (to draw the blocking screen on top of other apps). Both require the user to manually toggle a switch in Android Settings, so the plugin opens the correct Settings page and the Flutter side checks the result when the user comes back. The permissions onboarding screen walks the user through these before they can start a blocking session.

When the user starts a focus session with app blocking enabled, the Flutter side calls startBlocking with the list of blocked package names, the blocking level (soft, full, or aggressive), the override wait time in seconds, and the maximum number of allowed overrides. These values come from the coaching mode settings, so a Casual session might allow 5 overrides with a 15-second wait, while an Extreme session might allow 0 overrides entirely.

AppBlockerService launches as a foreground service with a persistent notification (required by Android for long-running background work). It polls the foreground app every 500 milliseconds using UsageStatsManager.queryUsageStats, finds the most recently used package by comparing lastTimeUsed timestamps, and checks it against the blocked list. System UI packages (launcher, dialer, settings) are whitelisted so the user can still interact with their phone normally.

The override system works through the overlay itself. If the user taps "I really need to use this app," a countdown timer starts (the duration comes from the coaching mode). They have to wait for the full countdown before a confirm button appears. Each override increments a counter and fires an event back to Flutter through the EventChannel, which the timer screen uses to track violation counts and apply penalties. Once the user hits the maximum override count, the override option disappears and they can only return to the focus session.

Getting foreground app detection to work across different Android versions took some trial and error. The UsageStatsManager approach works on all supported versions, but it requires the Usage Stats permission which users have to grant manually through Settings. The current polling approach at 500ms intervals is a compromise between responsiveness and battery usage. Faster polling catches app switches sooner but drains battery, and anything slower than one second feels laggy in testing.

4.3.3.3 Home Screen Widgets

The widget system provides two Android home screen widgets: a small 2x2 widget and a wider 4x2 widget. Both display the user's companion sitting inside a blanket fort scene, with the streak count and level overlaid as semi-transparent badges.

The architecture follows Android's AppWidgetProvider [15] pattern. SproutlyWidgetProvider.kt contains the WidgetData class that reads companion state from SharedPreferences, a WidgetRenderer object with the rendering logic, and two provider subclasses (SproutlySmallWidgetProvider and SproutlyMediumWidgetProvider) that Android calls when widgets need updating. WidgetPlugin.kt bridges Flutter to this system through the same MethodChannel pattern used by the app blocker.

Each widget has three visual states. The idle state shows the cozy blanket fort background with fairy lights lit, cushions arranged neatly, books stacked, and a steaming mug, with the companion in the center and streak/level badges in the corners. The session state uses a green-tinted version of the same scene and replaces the badges with a countdown or countup timer. The sleeping state triggers when the companion mood is set to "sleeping," which happens automatically when the user has not completed a session in over 24 hours. When sleeping is active, the widget switches to a dark background where the fairy lights are off, the cushions are scattered, and the mug is tipped over. The sleeping widget shows "Fort collapsed!" with a "tap to rebuild" message.

Widget backgrounds are pre-rendered PNG illustrations stored in drawable-xxhdpi. The companion images are also static PNGs, exported from the Rive animation files by a temporary utility screen that captures single frames using RenderRepaintBoundary.toImage at 4x pixel ratio for crisp 512x512 output. Each companion has an awake and sleeping variant, giving 12 companion PNGs total across the six available companions. The Kotlin code resolves the correct companion drawable at runtime by constructing the resource name from the companion type and sleep state (for example, companion_fox_sleeping), falling

back to the fox if the selected companion's image is not found (however, this case won't occur in the app as the user is forced to choose a companion throughout the usage).

The text overlays use RemoteViews, which is the only way to build Android widget layouts. RemoteViews has a limited set of supported view types (no custom views, no RecyclerView, no plain View elements), so the layouts use only FrameLayout, LinearLayout, ImageView, and TextView. The background scene, companion, and text all stack as layers inside a root FrameLayout. State changes (switching backgrounds, showing or hiding the timer area, changing text colors for the dark sleeping state) are handled through setImageViewResource, setViewVisibility, and setTextColor calls on the RemoteViews object.

When the Flutter side updates companion data or session state, it calls updateWidgetData through the MethodChannel with a map of changed fields. The plugin writes these to SharedPreferences and then calls WidgetRenderer.refreshAll, which iterates over all active widget instances of both sizes and re-renders them. Android also calls onUpdate periodically (set to every 30 minutes in the widget info XML), which serves as a fallback refresh in case a MethodChannel update was missed.

4.3.3.4 Persistent Focus Notification

During focus sessions, a persistent notification appears on the lock screen showing the session timer, task name, and a progress bar for pomodoro sessions. This notification cannot be swiped away by the user, so the timer stays visible even when the phone is locked.

FocusNotificationService manages this using a Handler that posts a Runnable every second. Each tick recalculates the elapsed or remaining time from the stored start timestamp, formats it, and updates the notification through NotificationManager.notify with the same notification ID, which replaces the previous notification in place without creating a new one. The notification channel is set to IMPORTANCE_LOW so it does not make a sound on each update, and setOnlyAlertOnce is set to true as an extra precaution.

The notification is started and stopped through the WidgetPlugin MethodChannel (startFocusNotification, pauseFocusNotification, stopFocusNotification). The Flutter WidgetService calls these automatically alongside the home screen widget updates, so starting a session in the app creates both the widget update and the lock screen notification in a single flow.

On Android 13 and above, apps need explicit user permission to post notifications. The WidgetPlugin implements ActivityAware to get access to the Activity context needed for the runtime permission dialog. The permission is requested once when the user first opens the home screen after installing the app. If the user denies it, the focus notification simply does not appear, but the rest of the app works normally. On Android 12 and below, notification permission is granted by default, so no dialog is shown.

4.3.4 Swift

4.3.4.1 General Setup

The iOS-native layer is implemented in Swift and bridges into the Flutter application through platform channels. Two method channels are registered at app launch inside AppDelegate.swift: com.sproutly/screen_time, which handles notification scheduling and focus session lifecycle, and com.sproutly/widget, which handles home screen and lock screen widget data updates. Both channels are initialized in application(_:didFinishLaunchingWithOptions:) and communicate with Flutter via FlutterMethodChannel, receiving method calls and returning results asynchronously on the main thread. All pending and delivered notifications belonging to Sproutly are cleared on every app launch to prevent ghost notifications from previous sessions persisting in the notification center.

4.3.4.2 Focus Session Lifecycle

The Swift layer maintains the authoritative state of the focus session for the purpose of notification gating. Four boolean flags track session state: whether a session is active (isSessionActive), whether the current phase is a break (isOnBreak), whether the timer is paused (isTimerPaused), and the current coaching mode and task name. Flutter calls startFocusSession when a focus phase begins, startBreak when a break phase begins, stopFocusSession when the session ends entirely, and setTimerRunning on every pause and resume. This separation ensures that distraction notifications are only armed during active focus phases, leaving the app during a break or while the timer is paused does not trigger any penalty notifications.

4.3.4.3 Distraction and Penalty Notifications

When the app enters the background (applicationDidEnterBackground), Swift checks all three session flags before scheduling any notifications. If the session is active, not on break, and the timer is running, it schedules a set of distraction notifications whose intensity is determined by the coaching mode. The casual mode schedules two follow-up reminders ten minutes apart, standard schedules four at seven-minute intervals, harsh schedules six at three-minute intervals, and extreme schedules ten reminders one minute apart. An immediate nudge fires after three seconds if the distraction_detected rule is enabled, and follow-up reminders are only scheduled if distraction_followup is enabled. Both rules are passed from Flutter as a list of enabled rule identifiers when the session starts. When the app returns to the foreground (applicationWillEnterForeground), all pending Sproutly notifications are cancelled immediately. A separate notifyTimerPausedTooLong method schedules a single reminder two minutes after the timer is paused, only during focus phases, which is cancelled automatically when the timer resumes.

4.3.4.4 Home Screen and Lock Screen Widgets

The widget layer is implemented as a separate iOS app extension target (SproutlyWidget) using the WidgetKit [14] framework. Data is shared between the Flutter app and the widget extension through a shared UserDefaults suite (group.com.sproutly.widget), which both targets can read and write via the App Group entitlement. Flutter writes companion data (streak, level, mood, companion type) and session timer data (active state, adjusted start timestamp, total seconds, pomodoro flag, task name) through the com.sproutly/widget channel, which Swift writes directly into the shared defaults and then calls WidgetCenter.shared.reloadAllTimelines() to trigger a widget refresh.

The widget supports three display families: small (systemSmall), medium (systemMedium), and lock screen (accessoryRectangular). The small widget shows the companion image, a live countdown or elapsed timer during sessions, and the streak and level badge when idle. The medium widget splits into a companion panel on the left and a stats panel on the right, showing a live timer and task name during sessions or streak and mood when idle. The lock screen widget shows a live timer and task name during active sessions, and streak and level when idle.

Timer accuracy is maintained through an adjusted start timestamp strategy. Rather than storing the raw start time, Flutter computes an adjusted start as now minus already elapsed seconds before writing to shared defaults. This means the widget's timer display, which simply computes now minus adjustedStart for countups or adjustedEnd minus now for countdowns, remains correct across pause and resume cycles without any special pause state handling in Swift. When the timer is paused, Flutter sets session_active to false, which causes the widget to display the idle companion view rather than a frozen timer. The widget timeline policy refreshes every second during active sessions and every thirty minutes when idle, balancing live accuracy against battery impact.

5. Test Cases and Results

T-1: Non-Recurring Calendar Task Creation

Test ID	T-1	Category	Functional
Objective	This test case is to verify that a user can successfully create a one-time, non-recurring task in the calendar without conflicts.		
Steps	1. Open the Sproutly mobile application and navigate to the Calendar/Scheduling module. 2. Click on the "Create New Task" option. 3. Enter a valid Task Name, a specific Date, and a Duration (e.g., "Study for CS465", October 20th, 2 hours). 4. Click 'Confirm' or 'Save' to create the task.		

	5. Verify the task appears on the specified date in the calendar view.
Expected	The task should be successfully created and displayed in the calendar. No error message should be triggered since there is no scheduling conflict.
Date-Result	27/04/2026 - Passed

T-2: AI Based Task Breakdown

Test ID	T-2	Category	Functional
Objective	This test case is to verify that the AI module can successfully break down a large task into manageable subtasks.		
Steps	1. Navigate to the Tasks page and select the option to add a new task. 2. Enter a complex, long-duration task (e.g., "Finish Senior Design Report"). 3. Select the "AI Breakdown" or "Split into subtasks" option. 4. Review the list of subtasks generated by the AI module. 5. Confirm and save the subtasks to the task list.		
Expected	The AI should generate a clear, ordered list of smaller steps (subtasks) based on the description. These subtasks should be automatically added under the parent task.		
Date-Result	27/04/2026 - Passed		

T-3: Detection and Prevention of Overlapping One-Time Tasks

Test ID	T-3	Category	Functional
Objective	This test case is to verify that the system detects and prevents overlapping tasks in the calendar.		
Steps	1. Navigate to the Calendar page where a task already exists for a specific time (e.g., 10:00 AM to 11:00 AM). 2. Attempt to create a new one-time task. 3. Set the time for the new task to overlap with the existing one (e.g., 10:30 AM to 11:30 AM). 4. Click 'Confirm' to save the new task.		
Expected	The system should detect the conflict, prevent the task from being created or warn the user about the conflict so that the user can reschedule, and display an appropriate error message to the user.		
Date-Result	27/04/2026 - Passed		

T-4: Pomodoro Timer Input Validation

Test ID	T-4	Category	Functional
Objective	Verify that the pomodoro timer can't be initialized with negative, empty, or extremely large values.		
Steps	1. Navigate to the Timer page after selecting a task. 2. Select 'Pomodoro Timer' as the timer of choice. 3. Set the fields to negative values, no value, or extremely large values. 4. Attempt to proceed to the next page.		
Expected	The system should detect the invalid input, and block the user from going to the page after the timer initialization page.		
Date-Result	27/04/2026 - Passed		

T-5: Focus Session Reminders

Test ID	T-5	Category	Functional
Objective	Verify that the application sends reminders when a focus session is ongoing.		
Steps	1. Start a focus session after selecting a task and setting up the timer. 2. Start the focus session timer. 3. Exit the application and open a different application whilst the timer is still running. Note that the app must still be open in the background.		
Expected	After a specified amount of time, a notification should pop up reminding the user about their ongoing focus session.		
Date-Result	27/04/2026 - Passed		

T-6: Upcoming Task Reminders

Test ID	T-6	Category	Functional
Objective	Verify that the application sends reminders for upcoming tasks.		
Steps	1. Navigate to the Calendar page and create a task. 2. Set the task's deadline to be 24 hours and 1 minute from now.		
Expected	The system should automatically send a reminder for a task when that task has 24 hours remaining.		
Date-Result	27/04/2026 - Passed		

T-7: Proof of Work Upload Flow

Test ID	T-7	Category	Functional
Objective	This test case is to verify the success path from completing a focus session through uploading a proof of work.		
Steps	1. Start a focus session in Standard Mode (1x XP). 2. Set the timer to run for 60 seconds. 3. Tap the “Complete Session” button. 4. Take a photo of the completed work. 5. Upload the proof for verification. 6. Read the verification result and session summary.		
Expected	The proof upload screen opens. After submission, the verification result screen shows the AI confidence score. The session summary displays XP earned. Companion XP increases accordingly.		
Date-Result	27/04/2026 - Passed		

T-8: XP Multiplier Scaling

Test ID	T-8	Category	Functional
Objective	This test case is to verify that the XP multiplier scales correctly among different coaching modes.		
Steps	1. Complete a 60-second session in Casual mode (0.8x). Note the XP earned. 2. Complete a 60-second session in Extreme mode (2.0x). Note the XP earned. 3. Compare the two values.		
Expected	After an extreme mode session, the user should earn approximately 2.5 times more than a casual mode session, such a result will confirm the correctness of the XP multiplier.		
Date-Result	27/04/2026 - Passed		

T-9: Companion Evolution

Test ID	T-9	Category	Functional
Objective	This test case is to verify that the companion evolves when the user completes a required number of tasks.		
Steps	1. Open the Sproutly application and navigate to the tasks screen. 2. Complete the required number of tasks needed for companion progression. 3. Open the companion screen from the home page.		

	4. Observe the companion's stage.
Expected	The companion should evolve to the next stage once the required number of tasks is completed, and the new stage should be displayed to the user.
Date-Result	28/04/2026 - Passed

T-10: Companion Mood Change

Test ID	T-10	Category	Functional
Objective	This test case is to verify that the companion's mood changes based on user activity.		
Steps	1. Log in to the Sproutly application. 2. Complete several tasks or a focus session to trigger a positive interaction. 3. Navigate to the home and companion screens. 4. Observe the companion's mood and expression.		
Expected	The companion's mood should update to a happy state after positive user activity.		
Date-Result	28/04/2026 - Passed		

T-11: Companion Progress Persistence

Test ID	T-11	Category	Functional
Objective	This test case is to verify that the companion's progress is saved between sessions.		
Steps	1. Log in to the Sproutly application. 2. Complete tasks that affect companion progress. 3. Log out of the application. 4. Log back into the application. 5. Navigate to the companion screen.		
Expected	The companion's stage, progress, and achievements should remain the same after logging back in.		
Date-Result	27/04/2026 - Passed		

T-12: Account Deletion

Test ID	T-12	Category	Functional
Objective	This test case is to verify that a user can successfully delete their account.		
Steps	1. Open the Sproutly app and navigate to the Account Settings screen. 2. Select the "Delete Account" option.		

	3. Confirm the deletion request. 4. Attempt to log in again with the same credentials.
Expected	The user account should be permanently deleted, and the system should not allow login with the deleted credentials.
Date-Result	28/04/2026 - Passed

T-13: Account Creation

Test ID	T-13	Category	Functional
Objective	This test case is to verify that a user can create a new account with valid information and access the application after successful registration.		
Steps	1. Open the Sproutly app and navigate to the sign up screen. 2. Enter valid registration information. 3. Tap the “Register” button. 4. Complete the “Personality Test” screen. 5. Confirm the app navigates to the dashboard screen after registration.		
Expected	The account is created successfully, the personality test is completed, and the user is routed to the dashboard screen.		
Date-Result	28/04/2026 - Passed		

T-14: Successful Log In

Test ID	T-14	Category	Functional
Objective	This test case is to verify that a registered user can log in with correct credentials and reach the dashboard screen.		
Steps	1. Open the Sproutly app and navigate to the sign in screen. 2. Enter a valid email and the correct password for an existing account. 3. Tap the “Sign In” button. 4. Confirm the app navigates to the dashboard screen after signing in.		
Expected	Login succeeds and the user is directed to the dashboard.		
Date-Result	27/04/2026 - Passed		

T-15: Log In with Incorrect Password

Test ID	T-15	Category	Functional
Objective	This test case is to verify that the system refuses invalid login attempts and shows a meaningful error message.		
Steps	1. Open the Sproutly app and navigate to the sign in screen. 2. Enter a valid email but an incorrect password.		

	3. Tap the “Sign In” button. 4. Observe the response shown by the application.
Expected	Login fails and the app displays a meaningful error message, no navigation occurs.
Date-Result	27/04/2026 - Passed

T-16: Recurring Calendar Task Creation

Test ID	T-16	Category	Functional
Objective	This test case is to verify that a user can create a weekly task with valid information when there is no scheduling conflict.		
Steps	1. Navigate to the Calendar screen. 2. Select “Create new task.” 3. Enter a valid task name, select a day of week, set a start time and a duration. 4. Tap the “Save” button. 5. Verify the task appears on the calendar in the correct slot.		
Expected	The recurring task is created successfully and displayed in the correct slot on the calendar with correct time and duration.		
Date-Result	27/04/2026 - Passed		

T-17: Calendar Task Input Validation

Test ID	T-17	Category	Functional
Objective	This test case is to verify that the system does not allow creating a calendar task with invalid inputs.		
Steps	1. Navigate to the Calendar screen and create a new task. 2. Leave the task name empty or do not select a day or time. 3. Tap the “Save” button. 4. Observe the response shown by the application.		
Expected	The task is not created and a meaningful error message is shown.		
Date-Result	27/04/2026 - Passed		

T-18: Edit Calendar Task

Test ID	T-18	Category	Functional
Objective	This test case is to verify that the user can edit an existing calendar task and the changes are shown correctly in the calendar.		
Steps	1. Navigate to the Calendar screen and find an existing weekly task.		

	2. Tap the task to open its details. 3. Tap the “Edit” button. 4. Change a field to a valid non-conflicting value. 5. Tap the “Save” button. 6. Verify the calendar shows the updated version of that task.
Expected	The task is updated successfully and the calendar view shows the new schedule.
Date-Result	27/04/2026 - Passed

T-19: Pomodoro Timer Pause/Resume Consistency

Test ID	T-19	Category	Functional
Objective	This test case is to verify that the Pomodoro countdown timer works without any time jumps during a start, pause or a resume operation.		
Steps	1. Start a Pomodoro session with any time interval you like (should be large enough to observe any possible time jump). 2. Tap the “Start” button to start the timer. 3. Observe the countdown for a few seconds for any unusual behaviour. 4. Tap the “Pause” button to stop the timer. 5. Wait a few seconds, then tap the “Resume” button. 6. Make sure that the timer continues from where it paused.		
Expected	The timer counts down from the time interval picked by the user. The pausing action freezes the countdown; the resuming action makes the countdown continue from where it left off without any time jumps.		
Date-Result	27/04/2026 - Passed		

T-20: Start Focus Session

Test ID	T-20	Category	Functional
Objective	This test case is to verify that a user is able to select an already created task from the task list to start a focus session after pressing the circular timer button at the bottom of the home screen.		
Steps	1. Log in and make sure at least one task exists in the system. 2. If no tasks exist, create one before starting a session. 3. Tap the circular timer button at the bottom of the home screen. 4. Tap on a task from the task list to select it. 5. Tap “Next” to proceed.		
Expected	All of the active tasks can be viewed through the focus task screen. A user can select the one that they desire and the selected task will be highlighted. Tapping “Next” will navigate the user to the Timer Setup screen.		

Date-Result	27/04/2026 - Passed

T-21: Successful Sign Out

Test ID	T-21	Category	Functional
Objective	This test case is to verify that the user can sign out successfully and won't be able to go back to the home page after signing out.		
Steps	1. Log in to the app. 2. Navigate to the Profile screen. 3. Press the "Sign Out" button. 4. Try to go back by pressing the back button or use the navigation methods provided by your OS.		
Expected	The user is signed out of the application and redirected to the login screen. Tapping the back button does not redirect the user to the home page.		
Date-Result	28/04/2026 - Passed		

T-22: Session Quit in Casual Mode

Test ID	T-22	Category	Functional
Objective	This test case is to verify that quitting a session without completion in Casual mode shows a simple confirmation dialog, and no XP penalties are applied.		
Steps	1. Note the companion's current XP value. 2. Start a focus session with Casual mode selected as the coaching mode. 3. Tap "Start" to start the timer. 4. Tap "Quit session" (before the countdown is over if Pomodoro is selected as the timer type). 5. Read the text inside the confirmation dialog. 6. Tap "Quit" to confirm it.		
Expected	A confirmation dialog appears with the text "Are you sure you want to end this session?" with "Keep Focusing" and "Quit" as response buttons, as the user wants to quit the session. After the user confirms, no penalty dialog appears. The user is redirected to the home page where he/she can observe that the companion's XP remained the same.		
Date-Result	28/04/2026 - Passed		

T-23: Session Quit in Standard Mode

Test ID	T-23	Category	Functional
Objective	This test case is to verify that quitting a session without completion in Standard mode shows a simple confirmation dialog, and -10 XP penalties		

	are applied.
Steps	1. Note the companion's current XP value. 2. Start a focus session with Standard mode selected as the coaching mode. 3. Tap "Start" to start the timer. 4. Tap "Quit session" (before the countdown is over if Pomodoro is selected as the timer type). 5. Read the warning message inside the confirmation dialog. 6. Tap "Quit" to confirm it. 7. Observe the penalty dialog and the final XP.
Expected	A confirmation dialog appears with the text "You will lose 10 XP if you quit now" with "Keep Focusing" and "Quit" as response buttons, as the user wants to quit the session. After the user confirms, a penalty dialog appears showing "-10 XP". The user is redirected to the home page where he/she can observe that the companion's XP is reduced by 10.
Date-Result	28/04/2026 - Passed

T-24: Focus Timer Background Persistence

Test ID	T-24	Category	Non-functional
Objective	This test case is to verify that the timer still runs and the session state persists when the app goes to the background and returns.		
Steps	1. Start a focus session and start the timer. 2. Get out of the app without closing the app (it should run in the background). 3. Wait 10 seconds. 4. Reopen Sproutly. 5. Observe the timer state.		
Expected	The timer continues running when the app is opened and returned to the foreground, the timer also accounts for the time elapsed while being in the background. The session is not lost or reset.		
Date-Result	28/04/2026 - Passed		

T-25: Prevention of Duplicate Navigation on Multiple Taps

Test ID	T-25	Category	Non-functional
Objective	This test case is to verify that tapping the "Complete Session" button rapidly multiple times does not cause the app to navigate to multiple proof upload screens.		
Steps	1. Start a focus session and let the timer run for a few seconds. 2. Press the "Complete Session" button multiple times quickly. 3. Observe the navigations.		

Expected	Only one proof upload screen appears, after returning from one of them another one does not appear.
Date-Result	28/04/2026 - Passed

T-26: Prevention of Duplicate Quit Confirmation Dialogs

Test ID	T-26	Category	Non-functional
Objective	This test case is to verify that tapping the “Quit Session” button rapidly multiple times does not cause the app to show multiple stacked confirmation dialogs.		
Steps	1. Start a focus session. 2. Before the session ends, tap the “Quit Session” button multiple times quickly. 3. Observe the number of confirmation dialogs displayed.		
Expected	Only one quit confirmation dialog appears.		
Date-Result	28/04/2026 - Passed		

T-27: Timer Pause on Quit Confirmation Dialog

Test ID	T-27	Category	Non-functional
Objective	This test case is to verify that the timer pauses when the quit confirmation dialog appears on the tap of the “Quit Session” button, and does not account for the time spent while the dialog is open.		
Steps	1. Start a focus session with Pomodoro countdown selected as the timer choice. 2. Tap the “Quit Session” button and note the timer value right at the moment of pressing the button. 3. Wait 10 seconds while the confirmation dialog is open. 4. Tap “Keep Focusing” to dismiss the dialog. 5. Compare the timer value to the noted timer value.		
Expected	The timer value after dismissing the dialog appears to be the same as the value before opening the quit confirmation dialog. The 10 seconds spent looking at the dialog are not counted.		
Date-Result	28/04/2026 - Passed		

T-28: Custom Pomodoro Time Configuration

Test ID	T-28	Category	Functional
Objective	This test case is to verify that the Pomodoro timer can be configured with any interval that the user enters and that the timer works according to the new custom values.		

Steps	1. Start a focus session. 2. During the timer setup, pick the Pomodoro (countdown) timer option. 3. Change the focus, short break, and long break fields to any interval you would like. 4. Change the "Long break after" field as well. 5. Tap the "Start Session" button.
Expected	The new entered values are applied during the countdown. Values are saved to SharedPreferences for the next session.
Date-Result	29/04/2026 - Passed

T-29: No XP for Zero Duration Focus Session

Test ID	T-29	Category	Functional
Objective	This test case is to verify that completing a focus session with zero focus time yields no XP no matter what the coaching mode and its XP multiplier is.		
Steps	1. Start a focus session in any coaching mode. 2. As soon as the timer starts, press the "Complete Session" button, not letting the timer run. 3. Upload the proof and get verified by AI. 4. Observe the XP earned in the session summary screen.		
Expected	In the session summary, the screen shows that 0 XP is earned. This is valid for any coaching mode.		
Date-Result	29/04/2026 - Passed		

T-30: Automatic Switch from Focus time to Short Break Period

Test ID	T-30	Category	Functional
Objective	This test case is to verify that the Pomodoro timer automatically switches to Short break time when the user's focus session time period ends.		
Steps	1. Start a focus session with Pomodoro (countdown) timer type selected. 2. Set the focus field to a value you would like keeping it minimal to not wait for a long time. 3. Tap "Start" and wait until the time period you entered reaches 00:00. 4. Observe the app's actions when the timer reaches 00:00.		
Expected	When the timer expires in a focus session, the app automatically switches to Short break.		
Date-Result	29/04/2026 - Passed		

T-31: Cooldown system in Harsh and Extreme Modes

Test ID	T-31	Category	Functional
Objective	This test case is to verify that starting a new focus session is not allowed right after quitting one in Harsh or Extreme mode due to the cooldown system being active.		
Steps	1. Activate the cooldown system by quitting a focus session in Harsh or Extreme mode. 2. While the cooldown banner is visible in the home screen, try to start a focus session by tapping the circular timer button at the bottom of the home screen. 3. Follow the starting a focus session process by selecting a task and configuring the timer settings. 4. Tap the “Start Session” button. 5. Read the feedback given on the screen.		
Expected	A red error message appears at the bottom of the screen with the text “Focus sessions locked for X more days”, and the user is not allowed to start a focus session, preventing the companion from evolving through sessions.		
Date-Result	29/04/2026 - Passed		

T-32: Blocking Overlays in Blocked Apps

Test ID	T-32	Category	Functional
Objective	This test case is to verify that a blocking overlay is shown when the user tries to enter a blocked app during a focus session and that the return button to return from that screen works. (This test is only valid for Android users as this functionality is only available for them)		
Steps	1. Add any distracting app to your blocked apps list. 2. Grant all required permissions through the app blocking onboarding screen that can be reached from the profile page. 3. Start a session in Standard mode. 4. Switch to the app that you added to the blocked apps list. 5. Observe the blocking overlay that comes up. 6. Tap “Return to Focus”.		
Expected	An overlay appears once the user tries to open an app that is present in the blocked apps list. This overlay makes sure that it is not allowed to enter the app at the moment with the companion’s image showing. Tapping the “Return to Focus” button returns the user back to Sproutly.		
Date-Result	27/04/2026 - Passed		

T-33: Personalized AI Recommendations

Test ID	T-33	Category	Functional
Objective	This test case is to verify that the AI module gives study recommendations customized to the user based on their study patterns derived from the session histories.		
Steps	1. Complete at least 10 focus sessions across different tasks and times of the day, as well as across different days of the week. 2. Navigate to the Insights page. 3. Read the AI generated recommendations. 4. Make sure that the data used to provide the recommendations actually reference the user's data. For instance, "You focus best during the evenings" or "You should consider using shorter sessions for Math related tasks".		
Expected	The AI module provides 2-3 recommendations based on the user's study patterns. The suggestions are relevant and they are derived from the user's data. If there is not enough data to reach an outcome, a message is shown saying "More sessions are needed to give recommendations."		
Date-Result	28/04/2026 - Passed		

6. Maintenance Plan and Details

6.1 System Monitoring and Issue Detection

The FastAPI backend exposes structured logging at the application level, capturing request lifecycle events, file processing outcomes, LLM call attempts and retries, and error traces via Python's standard logging module. On the mobile side, Firebase Authentication provides visibility into auth events, but crash reporting and client-side diagnostics are not currently instrumented. Integrating a tool such as Firebase Crashlytics would be a meaningful next step to capture unhandled exceptions and track client-side error rates in production.

6.2 Regular Updates and Improvements

The backend is organized into five independent routers: proof_of_work, weekly_review, companion, tasks, and notes, each owning its own endpoint definitions, prompt logic, and validation models. The proof_of_work router handles task verification (/pow/verify) and appeals (/pow/appeal), applying one of four coaching mode profiles (casual, standard, harsh, and extreme) which control both the verification confidence thresholds and the AI feedback tone. The weekly_review router generates end-of-week AI summaries from session and task data. The tasks router provides AI-powered task inference and subtask breakdown. The notes router handles note summarization, cross-note connection detection, and image/PDF OCR

structuring. The companion router exposes the companion state. This separation means AI logic, verification thresholds, coaching mode profiles, and prompt versions can be updated and redeployed on the backend without requiring a new mobile release. The model name and prompt version are surfaced in AI verification and review responses (VerificationResponse, AppealResponse, WeeklyReviewResponse), allowing the client to detect mismatches if needed; the tasks and notes endpoints do not currently expose these fields. Mobile updates, including UI changes, new companions, and Flutter-layer logic, follow standard Google Play and App Store review processes.

6.3 User Support and Documentation

The application includes a dedicated onboarding screen (PermissionsOnboardingScreen) that walks users through the three Android permissions required for app blocking: Usage Access, Display Over Other Apps, and the optional Accessibility Service used by Extreme coaching mode. The companion selection flow at first launch introduces users to the available companions before they reach the main shell. As the feature set grows, in-app contextual help covering coaching modes, the proof-of-work flow, and penalty mechanics is a planned addition to reduce onboarding friction for new users.

6.4 Scalability and Adaptation

The FastAPI backend is stateless at the application layer. The proof-of-work and weekly review endpoints hold no session state between requests, making horizontal scaling straightforward as usage grows. The companion endpoint currently returns static data and has no database dependency. On the client side, task and activity data is stored locally in a SQLite database (sproutly_master_db.db) via the sqflite package. As the user base grows, migrating task and activity data from local SQLite storage to a cloud-hosted database would enable true cross-device synchronization, and is planned as a future improvement. Adding new Rive companions requires adding a new .riv asset and updating the companion type enum and rendering switch in Flutter, while animation behavior within each companion's existing state machine is self-contained.

6.5 Data Management and Security

User identity is managed through Firebase Authentication, which handles credential storage and session tokens according to Firebase's security model. Data collection is limited to what is functionally required: focus session analytics, task records, and companion state, the majority of which is stored locally on the device. Users can delete their accounts through Firebase Auth's standard account deletion flow. Periodic dependency audits for both the Flutter and FastAPI stacks, along with security reviews of API input validation and file handling logic, will be conducted to maintain compliance with KVKK and GDPR as the application scales.

7. Other Project Elements

7.1. Consideration of Various Factors in Engineering Design

7.1.1 Constraints

This section discusses how public health, safety, security, welfare, and global, cultural, social, environmental, and economic factors influenced the project's detailed design decisions.

7.1.1.1 Public Health

Sproutly interacts with users' productivity habits and mental wellbeing, which affected the design to position the system as a supportive productivity tool rather than a medical system. Therefore, public health considerations are a crucial part of the design, especially in how the system delivers feedback. For this reason, AI outputs are limited to productivity coaching and do not provide health related advice. Public health considerations also shaped the Proof of Work feature, which is designed to support healthy productivity behavior rather than increase stress or guilt so it is treated as an optional mechanism and the user can disable this feature when needed.

7.1.1.2 Public Safety

Safety considerations affected the design of the focus and app restriction features by prioritizing user control. The system includes an emergency override option that allows users to access a blocked application when necessary to ensure users maintain control over their devices at all times. The restriction feature is designed to let users choose a mode, so restrictions reflect user preferences rather than forcing a fixed behavior for every user. Restriction modes are designed with safe default settings, and the application starts with Standard mode as the default option. Users can switch to a lighter or stricter mode depending on their preferred level of support. Actions that can significantly affect user experience, such as quitting a session, are designed to require an explicit warning and user confirmation. In addition, the system also is designed to avoid permanent or irreversible actions that could negatively affect users or their devices. Since Sproutly is a cross-platform application and there are differences between Android and iOS, restriction features are designed to have alternatives for when an operating system doesn't support them.

7.1.1.3 Public Security

User access is protected through Firebase Authentication, and user sessions remain active even after the app is closed and reopened so users stay signed in across app restarts. Communication with Firebase services is handled over HTTPS/TLS to protect data while being transmitted. Data collection is limited to what is required for core functionality, such as tasks, focus session configuration, and basic progress information, and unnecessary sensitive personal data is not collected.

7.1.1.4 Public Welfare

User welfare strongly influenced the design of gamification and penalty related features. The system is designed to use respectful and supportive feedback, avoiding shaming language. Gamification and penalty mechanisms are kept simple and customizable, allowing users to adjust or disable features such as Proof of Work based on their preferences so the system supports motivation rather than applying overly harsh penalties.

7.1.1.5 Global Factors

Global differences in working habits influenced the design of flexible coaching modes that users can choose from rather than enforcing a single productivity model. Also, global differences in daily schedules and time zones were considered during the design of the project, ensuring that the system supports flexible scheduling rather than fixed productivity timelines.

7.1.1.6 Cultural Factors

Cultural differences in productivity, discipline, and gamification affected the design by focusing on neutral language throughout the interface and feedback messages. The system avoids culture specific or potentially offensive phrasing and presents motivation in an acceptable language. In addition, multiple virtual companions are included, allowing users to choose a preferred companion rather than limiting users to one fixed companion.

7.1.1.7 Social Factors

Differences in users' responsibilities and daily routines affected the design by keeping task management and scheduling adaptable rather than relying on a single fixed workflow. To address this, the system supports multiple task categories, including study, personal, and office tasks, enabling users to organize their responsibilities according to their own routines and priorities. Similarly, focus support was designed to be adaptable through different timer options, such as Pomodoro and stopwatch modes, together with user configurable session durations, so that focus sessions can fit different working styles and environments.

7.1.1.8 Environmental Factors

Environmental factors have a limited direct impact since Sproutly is a software based system. However, efficiency considerations affected the design by reducing unnecessary background processing and avoiding excessive AI usage. The system keeps AI features optional and user triggered so resource usage remains low during regular use.

7.1.1.9 Economic Factors

Budget limitations affected the design by shaping system scope, architecture and technology choices. To manage costs, AI features are designed to be optional and used in a controlled way, rather than being required for core functionality due to the use of paid AI APIs. In addition, the project uses a paid Rive license, so animation and companion work is prioritized to make efficient use of the available tooling budget. Overall budget

considerations led to the use of open source tools and low cost technologies in the system architecture.

	Effect level (0-10)	Effect
Public health	8/10	Affected the design to support mental wellbeing by limiting AI to productivity coaching and keeping Proof of Work optional to avoid stress or guilt.
Public safety	6/10	The design ensures users can always exit restriction modes, and restrictions can be turned off or changed so users remain in control.
Public security	6/10	Influenced the design by prioritizing authenticated access and encrypted communication to protect user data.
Public welfare	9/10	Significantly influenced the design by prioritizing supportive language, user controlled settings, and customizable gamification.
Global factors	5/10	Influenced the design to support flexibility for various ways of working and time zones.
Cultural factors	4/10	Affected the system to use neutral language and different virtual companions.
Social factors	5/10	Affected the design by enabling adaptable focus sessions and notifications for different daily routines.
Environmental factors	3/10	Had limited impact, but influenced avoidance of unnecessary resource usage and kept AI usage lightweight and optional.

Economic factors	7/10	Affected the design through controlled AI usage and cost conscious selection of tools and platforms.

Table 1: Factors and Effect Levels

7.1.2 Standards

7.1.2.1 IEEE 830 Software Requirements Specification [16]

IEEE 830 is used as a reference for organizing and documenting the system requirements, especially for functional and non-functional requirements. This consistent requirements style makes it easy to link design decisions back to specific requirements.

7.1.2.2 UML 2.5.1 [17]

UML 2.5.1 is used as a reference for modeling the system architecture and detailed design. The UML diagrams prepared during the project represent use cases, subsystem decomposition, interactions between subsystems, and class structures in the overall system design.

7.1.2.3 ISO/IEC 25010 [18]

ISO/IEC 25010 is used as a reference to organize the project's non-functional requirements under standard software quality characteristics, such as usability, reliability, performance, maintainability, and scalability.

7.1.2.4 ISO/IEC/IEEE 12207 [19]

The overall project workflow is based on ISO/IEC/IEEE 12207 software life cycle principles, adapted to the scope of a student project. It follows a structured workflow across requirements, detailed design, implementation, and testing.

7.1.2.5 PEP 8 [20]

Python code written with FastAPI follows the PEP 8 style guide to maintain consistent formatting and readability. This improves maintainability by ensuring consistent coding style across the team and making future changes easier.

7.2. Ethics and Professional Responsibilities

The team aims to act in line with professional and academic integrity principles. Project work, code and documentation is produced by the group members and any ideas obtained from external sources will be used only when permitted by their licences and will be properly cited in the report or in code comments.

Within the team, tasks related to both documentation and implementation are divided in a fair way so that each member contributes meaningfully and shares the overall project workload. Collaboration is conducted in a respectful and transparent manner, and decisions are made collectively. In addition, communication with the project supervisor and course instructors is maintained in an honest, clear, and timely manner. Project progress and limitations are reported transparently.

7.3. Teamwork Details

Work was distributed by considering each team member's experience, and areas of interest. Academic responsibilities and implementation tasks were shared fairly among all team members, which helped the team work more efficiently and complete tasks more effectively.

7.3.1. Contributing and functioning effectively on the team to establish goals, plan tasks, and meet objectives

- Weekly group meetings were conducted on the weekend, both online and in-person, to review the current status of the project, discuss completed tasks, and plan the next steps.
- Both app development and report preparation workloads were distributed evenly among team members.
- The team maintained regular communication throughout the project using online meetings, messaging platforms, and GitHub. This helped members coordinate implementation tasks.

7.3.2. Helping creating a collaborative and inclusive environment

To create a collaborative and inclusive environment, we followed an inclusive working style prioritizing transparency, responsibility, and mutual support. We implemented the following strategies to maintain this environment:

- We defined clear and achievable milestones during each meeting, and made sure that the workload was distributed fairly.
- When possible, tasks were assigned by considering each team member's preferred application features or report sections.
- Timelines and deadlines were set realistically based on the complexity of each task. Tasks that required more design, implementation, or integration effort, such as companion animations and major feature development, were given longer completion periods.
- Whenever an important decision needed to be made, all team members were consulted, and these decisions were made through consensus.
- We have centralized our communication and workflow with different tools. Zoom for simultaneous workloads and online meetings, WhatsApp for real-time updates, and GitHub for version management and collaborative development.

7.3.3. Taking lead role and sharing leadership on the team

Every member was assigned a different work package to lead from the start of the project, which ensured that everyone took the lead role and the team to share leadership.

- Kağan Rehber is leading the frontend development, ensuring a design that everyone agreed on and that was accessible.

- Bilge İdil Öziş is leading the AI/ML development, focusing on building robust models and integrating intelligent features into the system.

- Bilgehan Tuğcu is leading the backend development and database integration.

- Mennatallah Abouelenin is leading the companion system, creating companions and animations, and integrating the animated companions into the project.

- Elif Ercan is leading the gamification system, setting up XP, and handling achievements as well as companion designs.

7.3.4. Meeting objectives

The overall aim of the project was to develop a functional cross-platform productivity application that combines task management, focus sessions, AI-assisted support, app restriction, and a gamified companion system. The initial project plan divided this objective into five main work packages: frontend development, backend and database development, gamification and companion development, AI/ML development, and testing. These work packages were used throughout the project to organize responsibilities, track progress, and evaluate whether the planned milestones were achieved.

Frontend Development
The main mobile interface was implemented using Flutter, including authentication screens, the home page, task management flow, timer setup, active focus session screens, pages related to profile, notes, and companion-related screens. The completed frontend supports the main user workflows and provides a consistent user experience across the application.

Backend and Database Development
Firebase Authentication was integrated for user registration and login. Data management was implemented using SQLite and SharedPreferences for tasks, notes, focus sessions, settings, weekly activity data, and progress information. The FastAPI backend was implemented and organized into routes for Proof of Work, weekly review, companion, tasks, and notes. It supports feature-specific backend logic and handles communication with external AI services when AI functionality is required.

Gamification and Companion Development
The application includes coaching modes ranging from Casual mode to Extreme mode, XP calculations, penalties, streak tracking and six different customizable virtual companions. These components were integrated into the main productivity workflow, allowing session outcomes such as completion, quitting, or override usage to affect XP, penalties, streaks, and feedback shown to the user.

AI/ML Development
The project prioritized AI-supported features that were most closely connected to the main user workflow, such as task breakdown, Proof of Work verification, and weekly review feedback and suggestions. These features were integrated into the application to reduce the user's planning effort, verify session outcomes, and provide personalized productivity suggestions.

Each work package was assigned to responsible team members, and progress was reviewed regularly through group meetings. This helped the team handle the frontend, backend, AI, and gamification tasks in an organized and consistent way while supporting teamwork. When implementation challenges occurred, such as integrating AI-supported features into the user workflow or handling platform differences in app restriction behavior, the team discussed these issues and reassigned tasks when needed. Overall, the project objectives were met at a satisfactory level, and the final version of Sproutly reflects the main goals defined in the initial project plan which are: supporting task management, schedule planning, focus sessions, app restriction, companion-based motivation, and AI-assisted productivity features in a single mobile application.

7.4 New Knowledge Acquired and Applied

While working on and developing Sproutly, the team gained new technical and design knowledge in different areas. Since the project combined mobile development, gamification, AI-supported features, platform-specific functionality, and documentation, team members had to learn and apply various tools during the implementation process. The knowledge gained during the project directly affected how the application was designed, implemented, tested, and documented.

7.4.1 Cross-Platform Mobile Development with Flutter and Dart

One of the main areas of knowledge acquired during the project was cross-platform mobile development using Flutter and Dart. Since not all team members were familiar with Dart at the beginning of the project, the team had to learn its syntax, widget structure, state management approach, and navigation logic while implementing the application. This knowledge was then used in the development of main screens such as the authentication pages, home screen, task management pages, timer setup screen, active focus session screen, profile screen, weekly review screen, notes screen, and companion pages.

Working with Flutter also helped the team understand how to design screens, manage screen transitions, store user preferences, and maintain a consistent interface across different parts of the application. These skills were important for creating a user friendly interface and for keeping the mobile application maintainable for more features to be added.

7.4.2 Companion Design and Animation with Rive

Before the project, the team had no experience with Rive and interactive animation workflows. The team members responsible for the Rive component learned how to design and animate virtual companions using Rive and used this knowledge in the companion system. During development, the team learned how to create companion assets, make animations, and use Rive state machines to trigger companion animations inside the application.

This knowledge was applied to the companion system, which is one of the main gamification elements of Sproutly. The team created six different companion options and integrated them into the application. Learning Rive also helped the team understand how animation design affects user experience and motivation, since the companion needed to feel supportive without distracting the user from the main productivity and focus workflow. The companion designs and possible animations needed for the companions were decided collaboratively by the whole team and then distributed among the team members responsible for the Rive component.

7.4.3 Platform-Specific Development for Android and iOS

Although Sproutly is mainly developed with Flutter, some features required knowledge of native mobile platforms. The team gained experience with platform-dependent development concepts in Android and iOS. Since Android and iOS provide different system permissions and restrictions, some features had to be implemented differently. This was especially important for app restriction behavior, widgets, and notifications. In cases where a feature could not be implemented in the same way on both platforms, alternative flows were designed. For example, some features related to restriction are more limited on iOS, so the application uses consequences based on the chosen coaching mode as alternative implementations.

For Android, the team worked on permission-based functionality such as usage access and overlay behavior for app restriction. For iOS, the team learned that some restriction features are more limited because of operating system rules, so alternative approaches had to be considered. This showed that developing in multiple platforms may require different implementations for the same feature on different operating systems.

7.4.4 AI Integration and LLM Based Features

Another important area of knowledge gained was the integration of AI-supported features into a mobile application. The team learned how to integrate LLM based services into

application features and applied this knowledge in features such as task breakdown, Proof of Work verification, weekly review feedback and suggestions, and note summarization.

Implementing these features helped the team understand how prompts, user input, and backend communication work together in LLM based features. The team also learned that AI features should be integrated in a controlled way so that they support the main purpose of the application instead of becoming separate or unnecessary additions. In Sproutly, AI was used mainly to reduce planning effort,verify submitted work after focus sessions, and provide personalized feedback.

7.4.5 Software Design, Documentation, and Project Communication

The project also improved the team's knowledge of software design documentation and communication during the project. Throughout the semester, the team prepared reports, diagrams, work packages, and test cases to present the system in a clear way. This required learning how to present architecture decisions, subsystem responsibilities, data flow, testing results, and engineering constraints clearly.

The team also gained experience in creating and updating diagrams such as use case diagrams, class diagrams, and subsystem diagrams. These materials helped the team structure the system design more effectively and made it easier to connect implementation work with project requirements. In addition, preparing documentation throughout the project helped the team understand the project's progress, and remaining tasks more clearly.

8. Conclusion and Future Work

The final implementation of Sproutly meets the project's main goal which is a cross-platform mobile productivity application that connects focus sessions with a gamified companion system, helping users build consistent work habits without relying on coercive blocking or rigid routines. Throughout the development process, the team implemented the full application stack: a Flutter-based mobile client for both Android and iOS, a FastAPI backend, platform-dependent features implemented with Kotlin and Swift and AI-based features supported by Claude Sonnet 4.5. All planned core features were delivered, including the focus timer with Pomodoro and chronometer modes, coaching modes with differentiated penalty systems, the animated Rive companion system, app restriction overlays on Android, Proof of Work verification, AI task breakdown, and personalized weekly feedback and recommendations.

Several improvements are planned for future development. The most significant is moving locally stored task and activity data to the cloud-hosted MySQL database to support synchronization across devices. On the companion side, the accessory system is designed to support future expansion. Additional unlockable accessories and new companion characters can be added in later versions of the application. The achievements system is also planned for expansion, with more milestone types and badges to reward a broader range of user

behaviors beyond session completion. In addition, expanding notifications to include more general recurring ones, such as daily task reminders, streak reminders, and mood check-ins, is planned for completion in the future, as the user preference UI is already in place. Another planned future improvement is schedule image parsing using OCR. This would function as another AI based feature that allows users to upload an image of their schedule. The system would then extract relevant tasks and events from the image and add them to the user's calendar. Another planned improvement is the visual evolution of companions. In the current version, the companion system tracks progress through XP and levels, but future versions can make this progress more visible by changing the companion's appearance as the user levels up. For example, companions could grow, unlock new visual stages, or gain new design details based on the user's consistency and completed focus sessions. This would make the progression system more noticeable and provide users with a stronger sense of achievement.

9. User Manual

9.1 Login Page

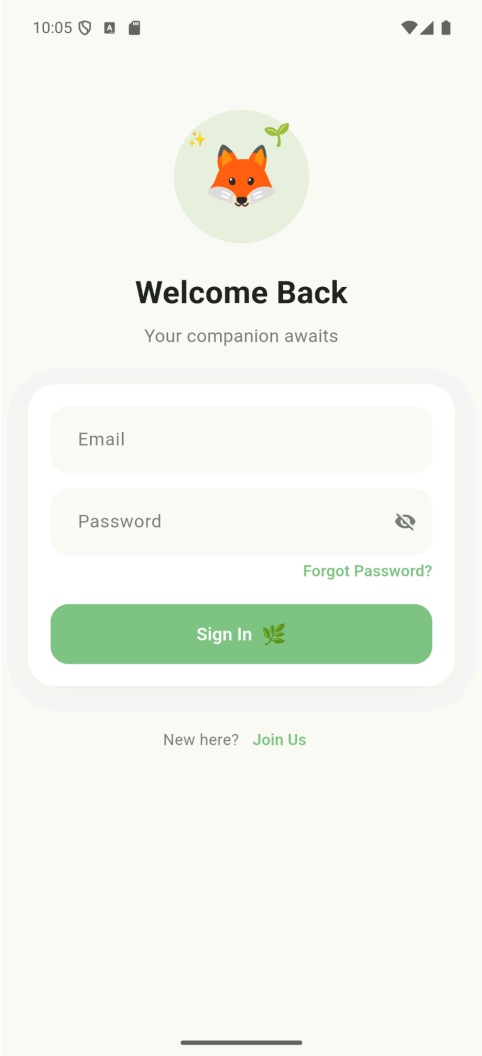

Figure 2: Login Page for Sproutly

9.2 Sign-up Page

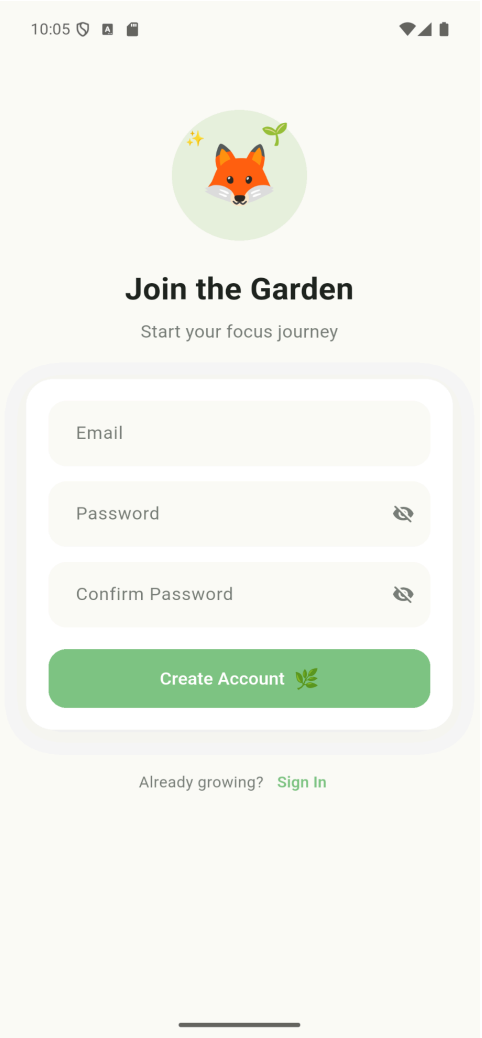

Figure 3: Sign-up Page for Sproutly

These pages are where the user logs in and signs up, respectively. The user can enter their e-mail and specify a password. Account creation is handled through requests to the Firebase cloud database. The backend logic and database do not allow for creating an account with the same e-mail.

9.3 Name Page

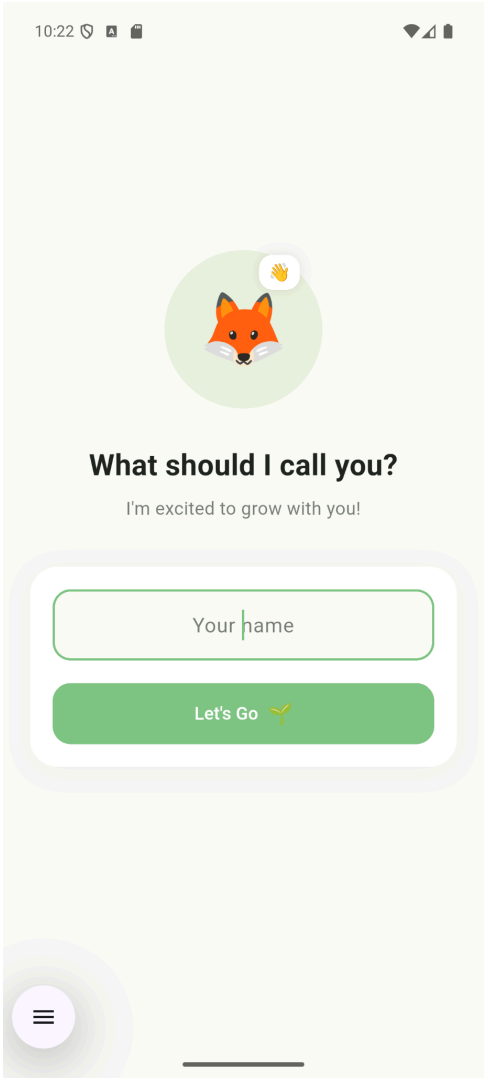

Figure 4: Name Page for Sproutly

9.4 Personality Test Page

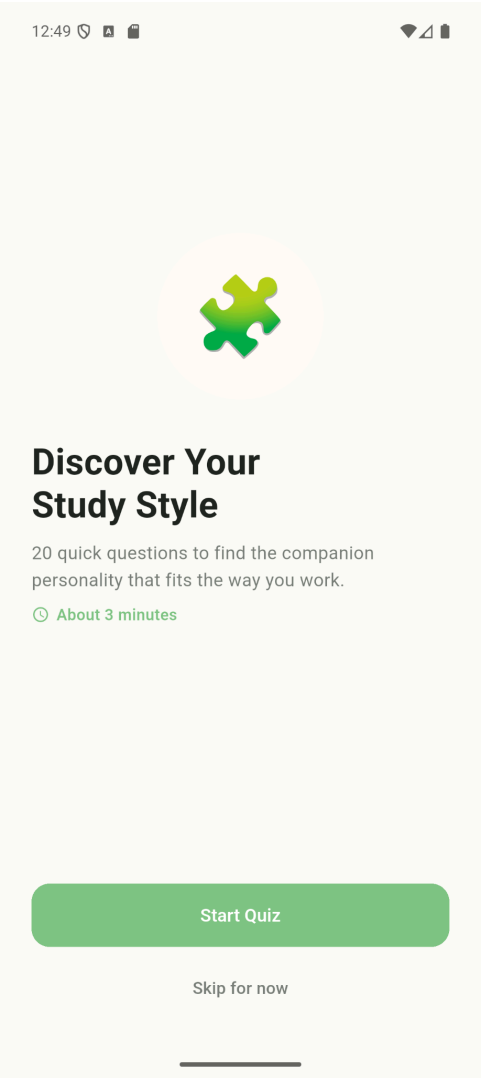

Figure 5: Personality Test Page for Sproutly

These pages are where the user can enter their name, and take an optional personality test. The user can take the personality test through the Settings Page (9.37), if they wish.

9.5 Personality Test Question Page

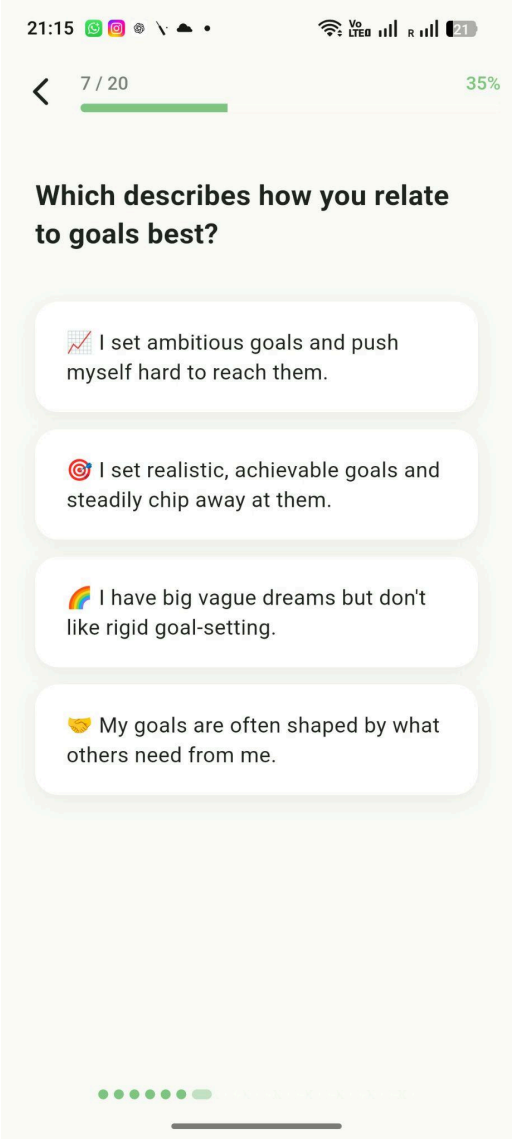

Figure 6: Personality Test Question Page for Sproutly

9.6 Personality Test Results Page

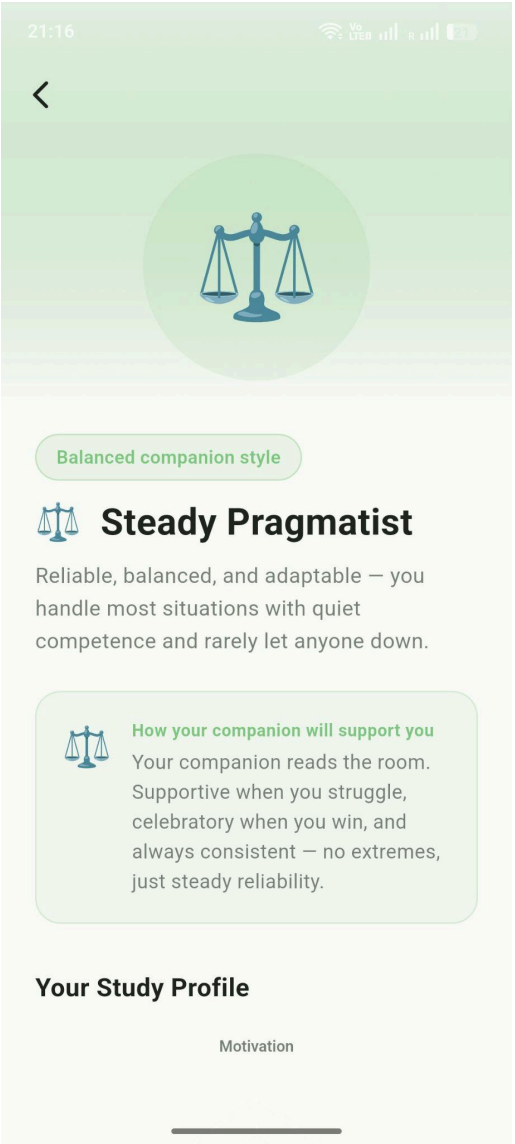

Figure 7: Personality Test Results Page for Sproutly

Each question on the personality test has multiple answers that the user can choose from. The personality test is created with the help of our innovation expert.

At the end of the test, the user can see their results, in which their personality type is shown.

9.7 Personality Test Results Page (Cont.)

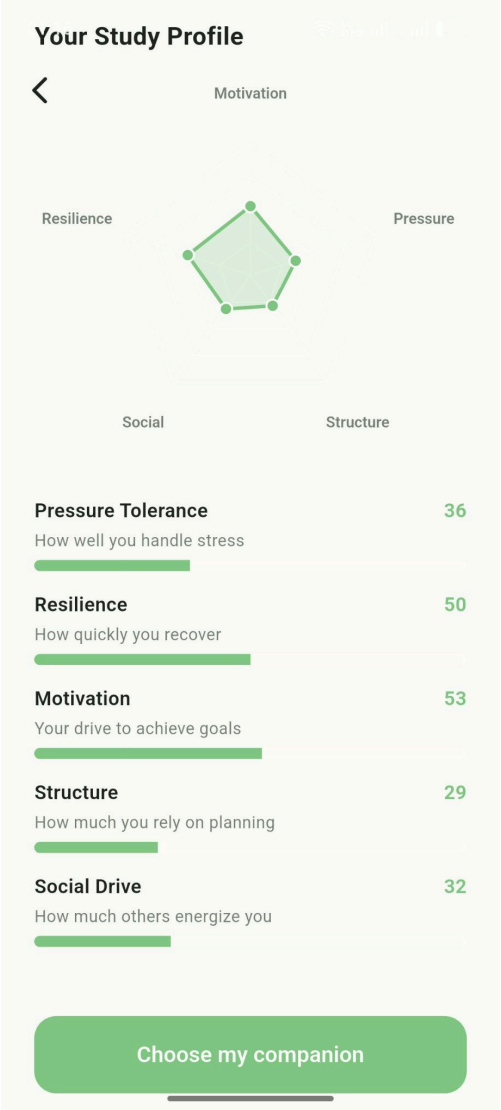

Figure 8: Cont. Personality Test Results Page for Sproutly

9.8 Companion Selection Page

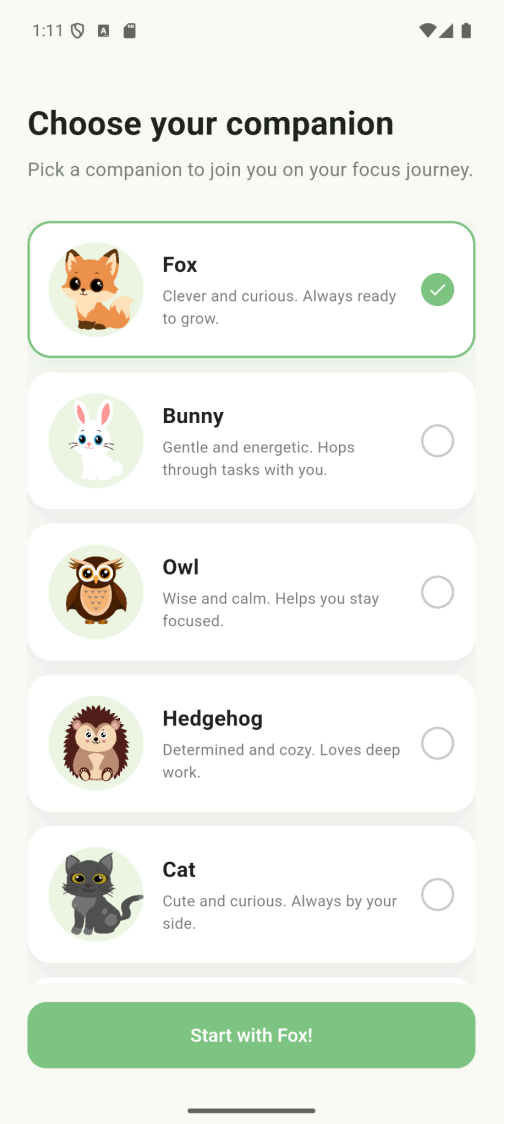

Figure 9: Companion Selection Page for Sproutly

Personality test results also show the user's study profile based on their answers.

Right before completing their registration, the user can select their companion; which will appear on the home page.

9.9 Home Page

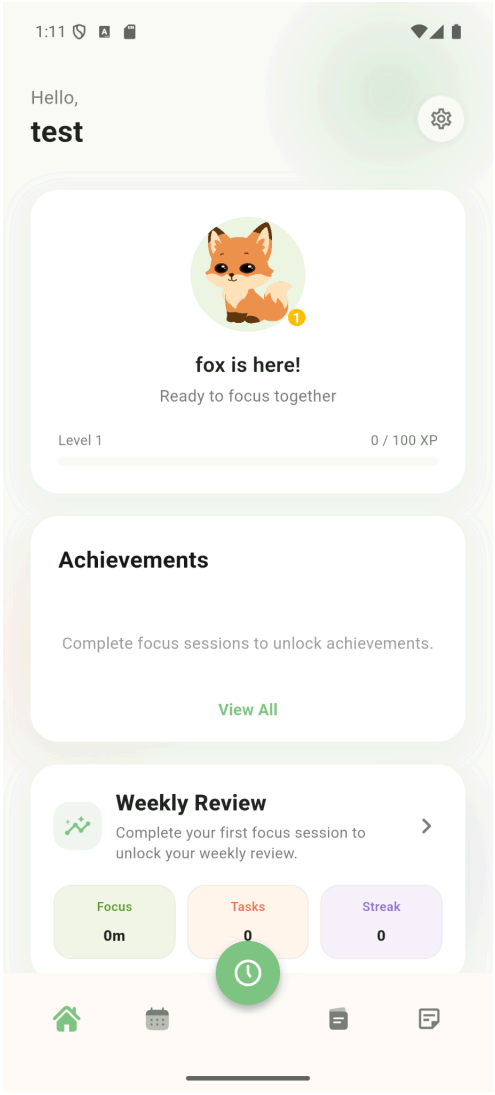

Figure 10: Home Page for Sproutly

9.10 Companion Page

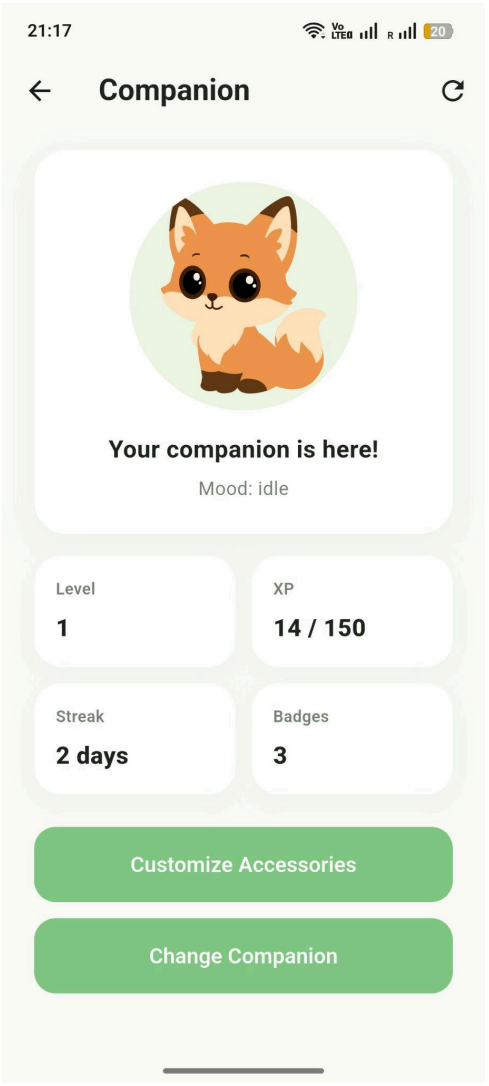

Figure 11: Companion Page for Sproutly

The home page includes a companion card that shows the selected companion, current level and XP. Below the companion card, the user can view achievements, weekly review preview and mood check-in feature. At the bottom there is a navigation bar where the user can navigate to the calendar page, timer setup for focus sessions, the tasks page, and the notes page.

The companion card routes the user to the Companion Page, in which the user can see their level and streak, XP they gained, and number of badges they have. Through this page they can also customize or change their companion.

9.11 Customize Accessories Page

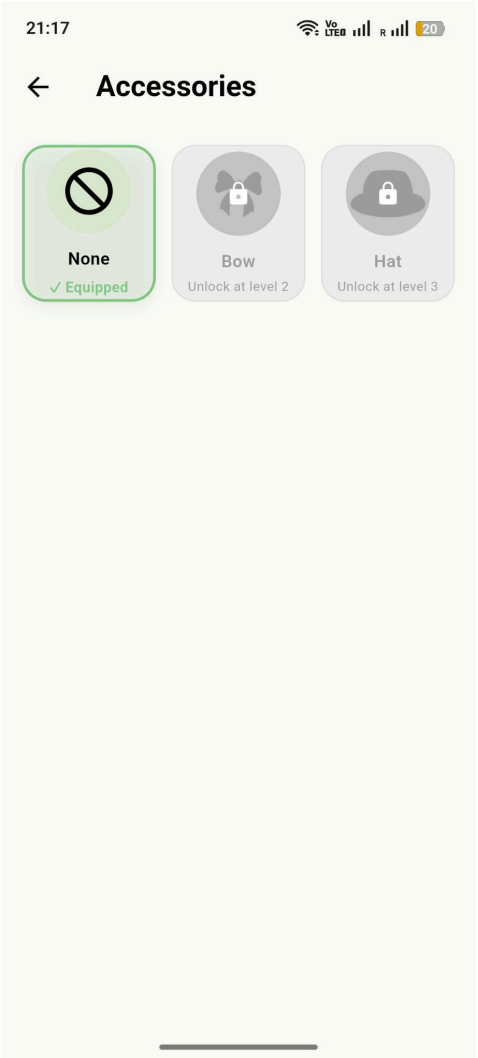

Figure 12: Customize Accessories Page for Sproutly

9.12 Calendar Page

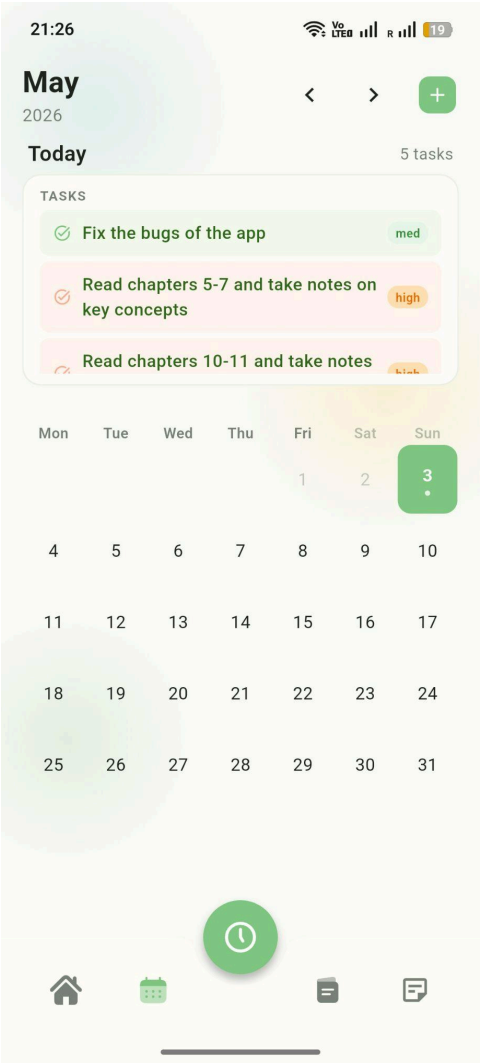

Figure 13: Calendar Page for Sproutly

The user can unlock the accessories and customize their companion after reaching a certain level from the Customize Accessories page.

The user can view their scheduled tasks and activities in the Calendar page. By clicking the '+' icon on the top right, the user can create new activities and tasks, which will appear on the calendar.

9.13 Calendar Page (Expanded)

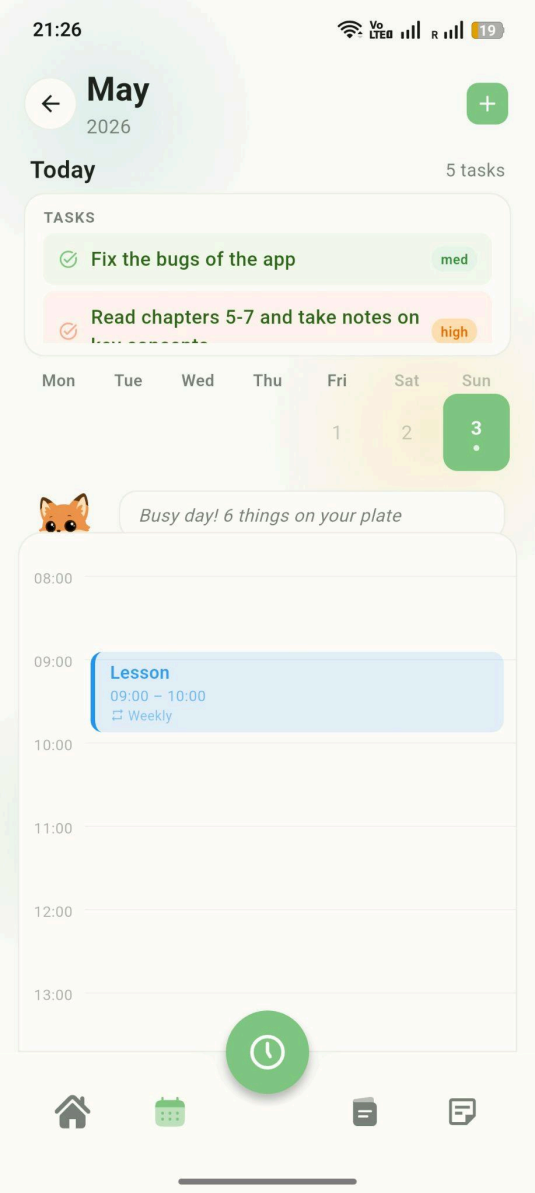

Figure 14: Expanded Calendar Page for Sproutly

9.14 My Tasks Page

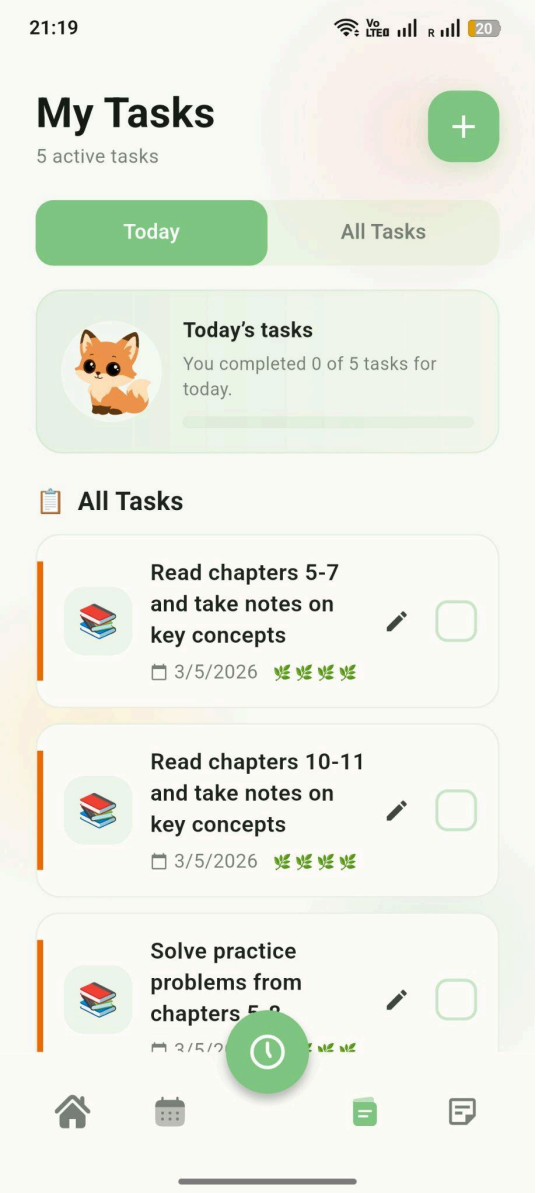

Figure 15: My Tasks Page for Sproutly

On the Calendar page, when clicked on a day, an hourly schedule is opened and shows the user's tasks.

The user can observe their tasks, the tasks' priority levels and deadlines through the My Tasks Page. There is also a companion card that shows how many tasks are completed that are due today at the top of the page.

9.15 Task Creation Page

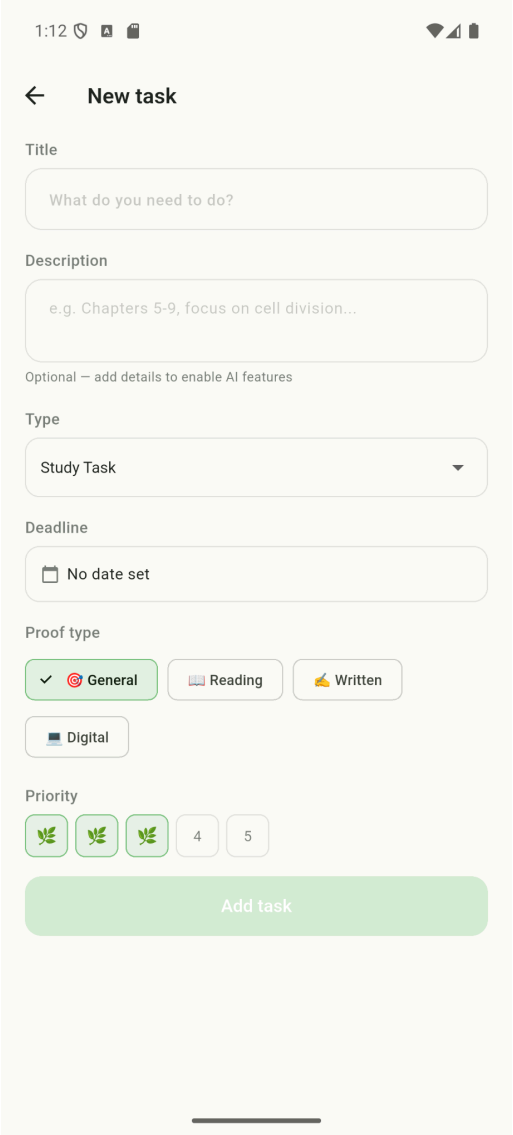

Figure 16: Task Creation Page for Sproutly

9.16 Task Creation Page (Cont.)

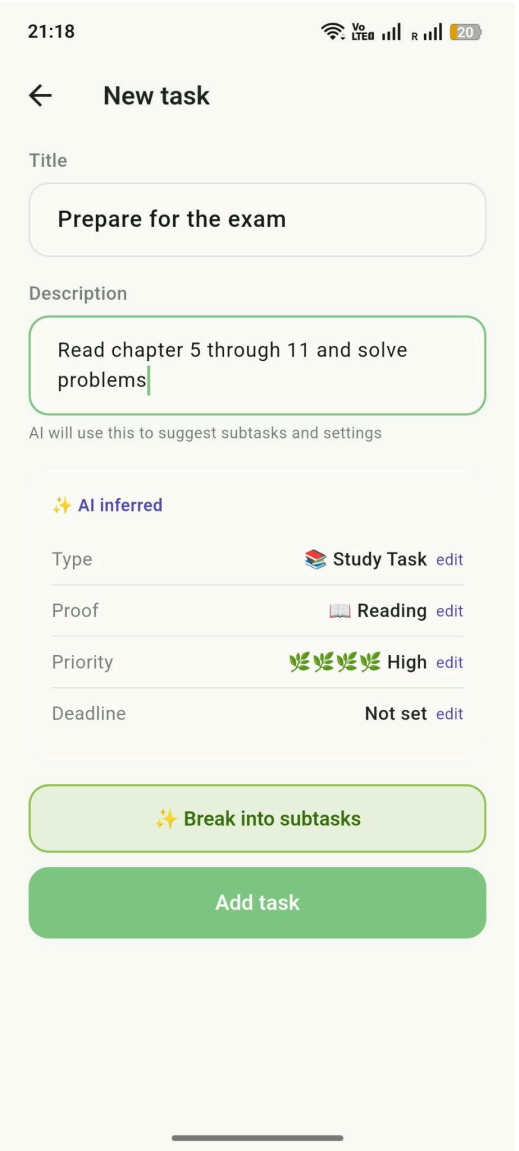

Figure 17: Cont. Task Creation Page for Sproutly

The task creation page is the interface through which the user creates one-time tasks. The user can enter a name, description, set a deadline; and specify parameters such as the type of task and its priority level.

If the user enters a description, the AI task breakdown feature will offer to break the user's task into subtasks.

9.17 Task Creation Page (Task Breakdown)

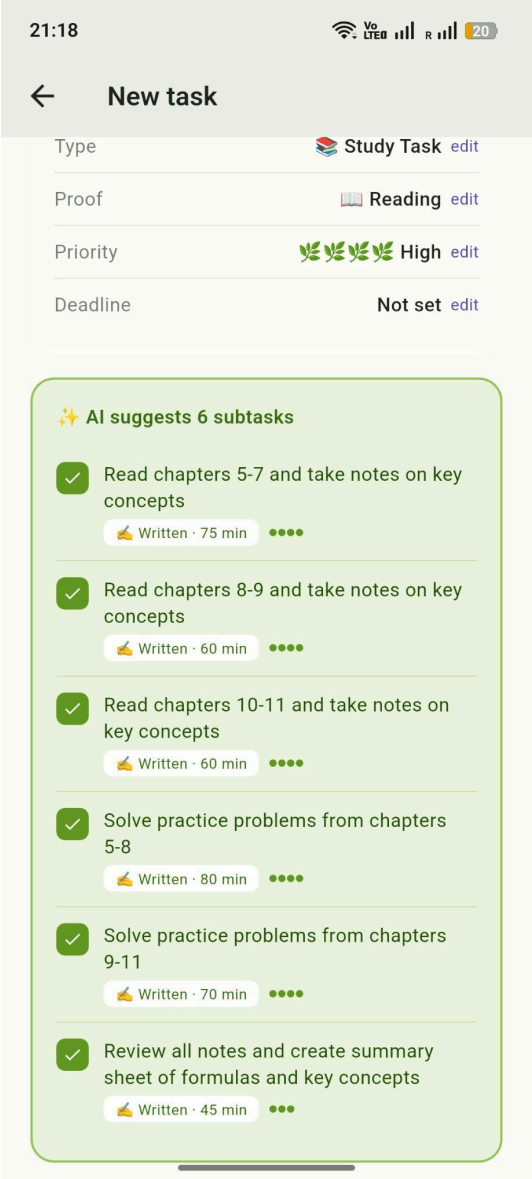

Figure 18: Task Creation (Task Breakdown) Page for Sproutly

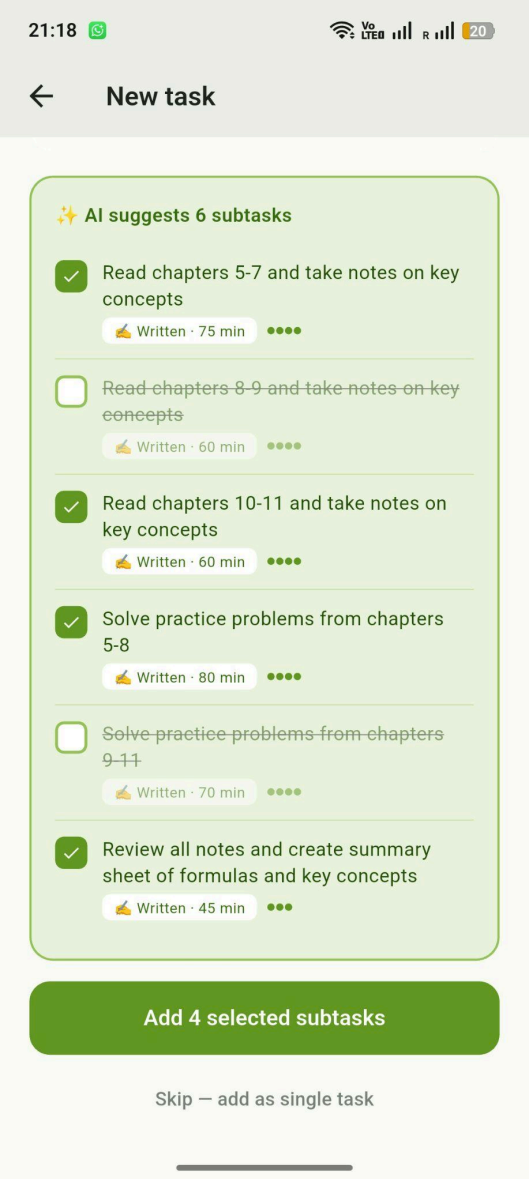

Figure 19: Cont. Task Creation (Task Breakdown) Page for Sproutly

When the tasks are too big to handle for a user, the user can write its description while adding a new task, the AI will infer the task's type (whether it is a study, office or a personal task), its proof type (whether it would be logical to send a written proof or a digital proof such as code), and the priority of the task. The user can change this information however they like by clicking the edit button next to it. The user can either add this task as is or choose to break down the task into subtasks. AI will suggest 3 to 6 subtasks with differing priorities and proof types. The user can pick whichever subtasks he/she will do. Moreover, they can also still add the task as a single task.

9.18 Event Creation Page

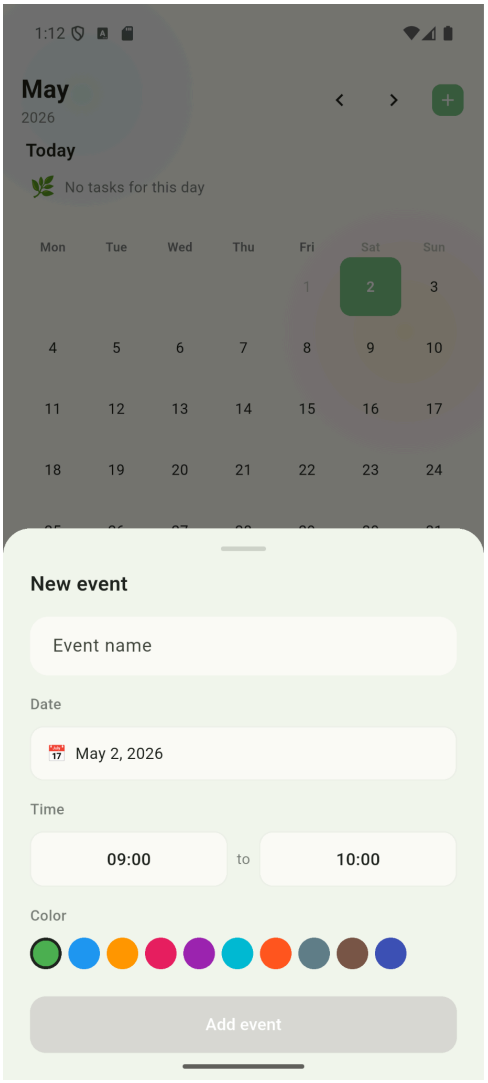

Figure 20: Event Creation Page for Sproutly

9.19 Recurring Activity Creation Page

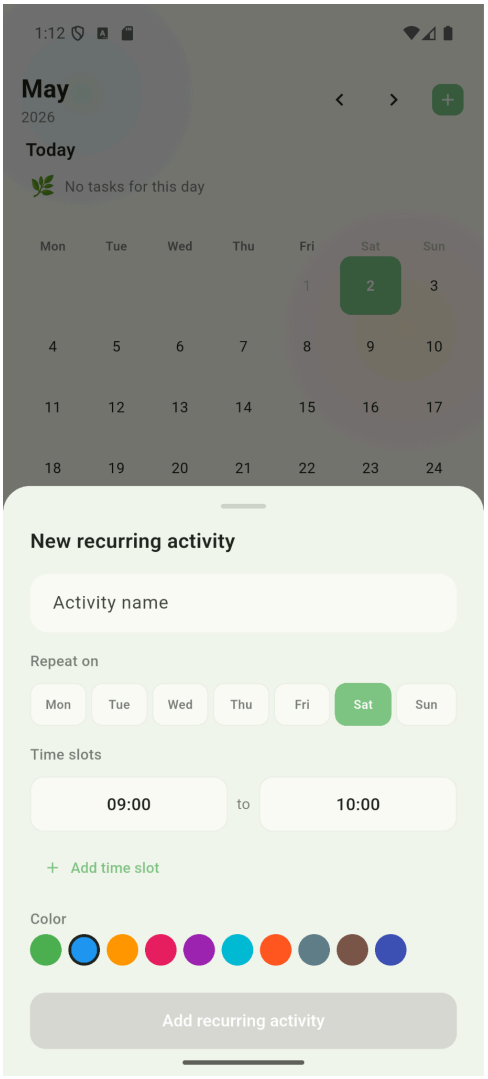

Figure 21: Recurring Activity Creation Page for Sproutly

The user can add a one-time event, such as an exam or a meeting, and specify its timeslot and timespan. Additionally, the user can create recurring activities, which repeat every specified day of the week, such as lectures. For both additions, the user can select a color and the task, event, or recurring activity will show up with the selected color on the specified date.

9.20 Focus Session Page

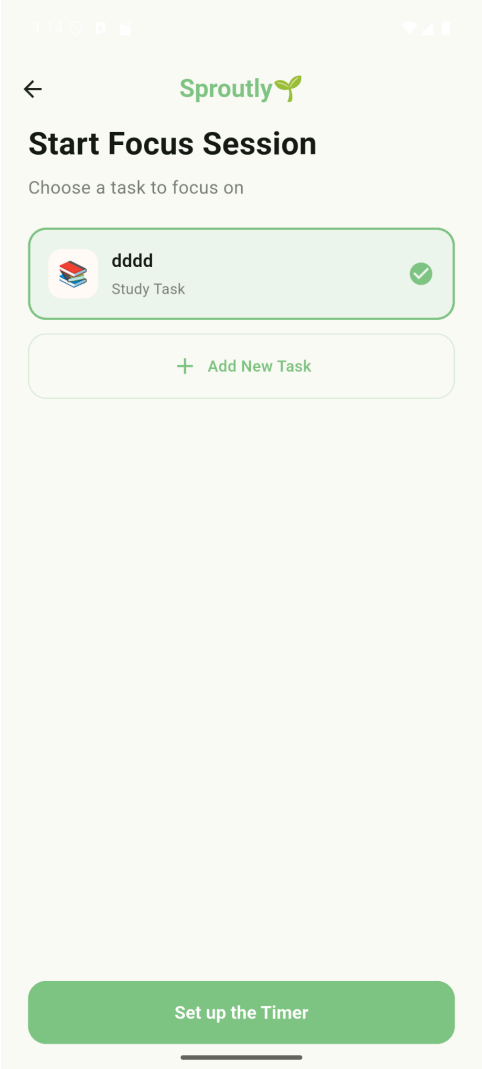

Figure 22: Focus Session Page for Sproutly

9.21 Timer Setup Page

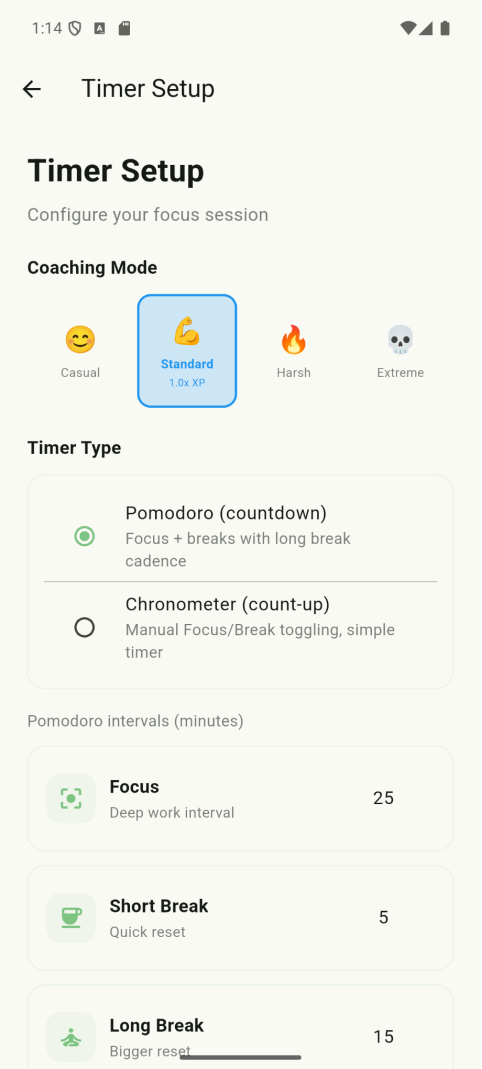

Figure 23: Timer Setup Page for Sproutly

By clicking the ‘clock’ icon on the bottom navbar, the user can navigate to the focus session page. From here, they can select a task to focus on, and proceed to the timer setup page by clicking ‘Set up the Timer’.

On the timer setup page, the user can specify the type of timer they would like to use, as well as the coaching level, which determines experience parameters for their companion. Selecting Pomodoro, the user can specify the parameters of a traditional pomodoro timer.

Switching the timer to Chronometer, the user will have access to a simple chronometer that counts-up, and laps whenever it is paused.

9.22 Timer Setup Page (Cont.)

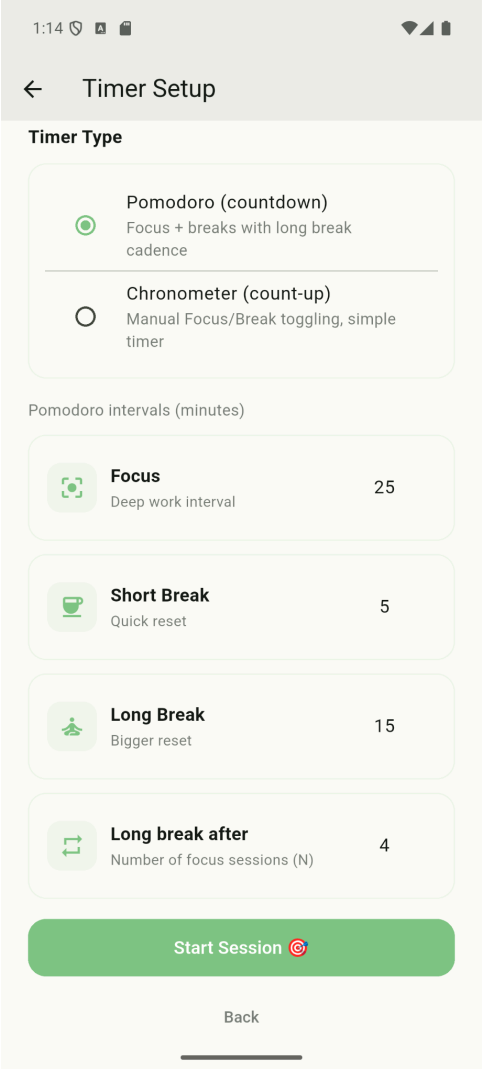

Figure 24: Cont. Timer Setup Page for Sproutly

9.23 Timer Page

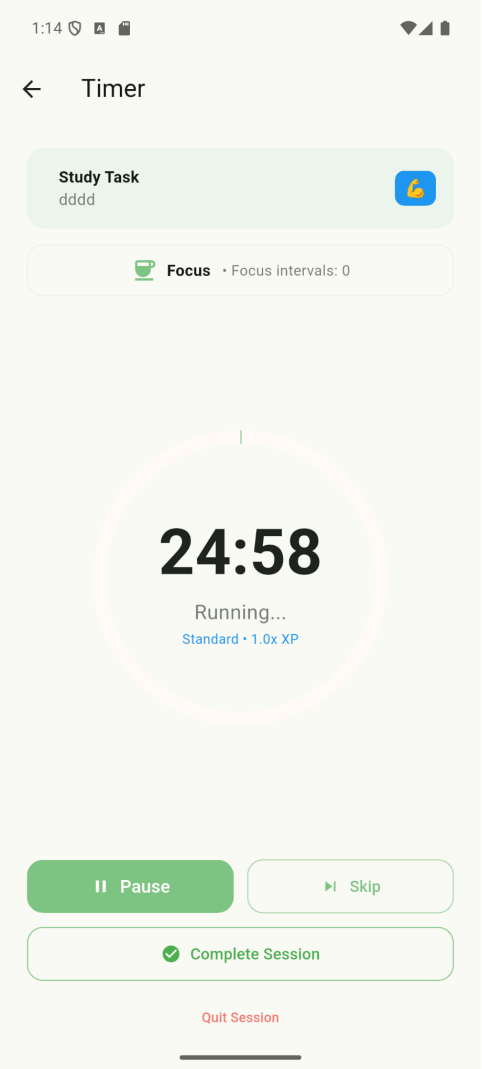

Figure 25: Timer Page for Sproutly

After clicking ‘Start Session’ on the timer setup page, the user will be led to the timer page. The user can pause the timer, skip, complete the session or quit the session. Skipping the timer causes it to go to the next time interval (e.g. choosing a 25 minute focus, 5 minute break period; if the user skips whilst the focus period is running, the timer will immediately go to the break period). Quitting the session incurs an experience penalty for the companion, which changes depending on the coaching mode selected in the Timer Setup Page (9.21).

9.24 Quit Session Companion Popup

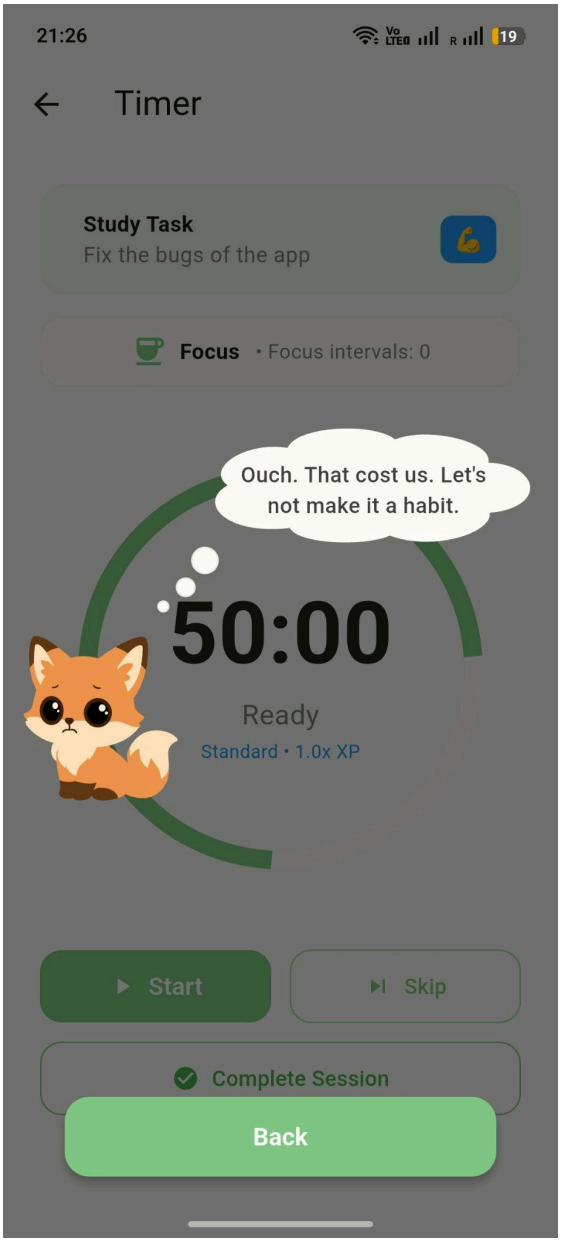

Figure 26: Quit Session Page for Sproutly

9.25 Proof Upload Page

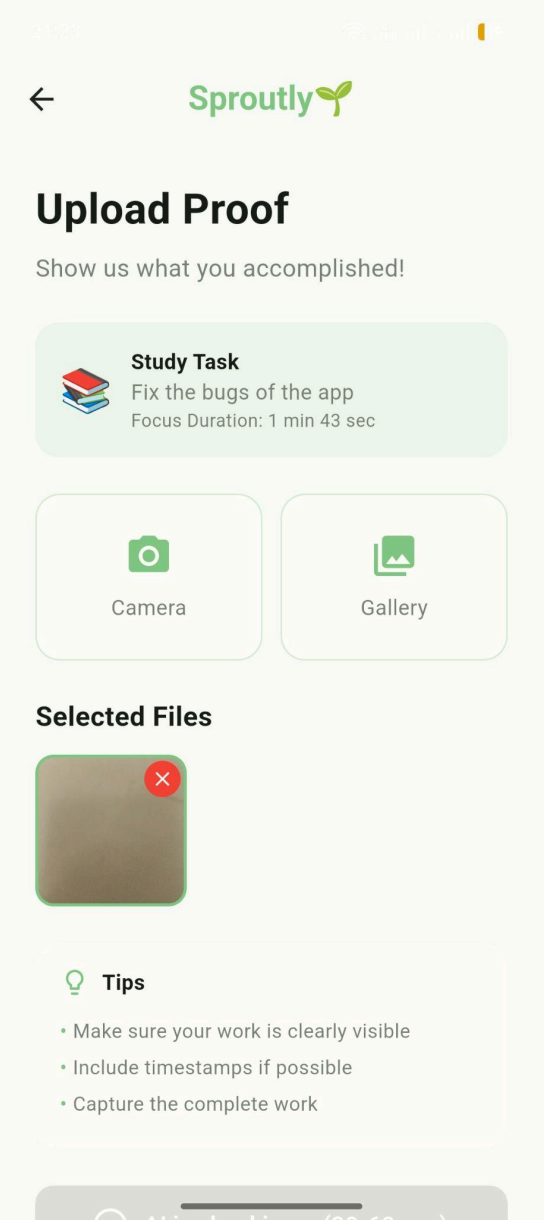

Figure 27: Proof Upload Page for for Sproutly

If the user quits a session, no XP will be gained and the companion will slide up to the screen in a sad animation showing disappointment. The user can face some penalties depending on the coaching mode. The back button under the companion dialog will navigate the user to the home page.

The user will need to upload proof after completing a session. This setting can be turned off from settings if the user desires a more lenient app. Also, the user can skip this option when this screen pops up, however, in that case it will be counted as the task is not completed and no XP has been gained.

9.26 Proof of Work Rejection Page

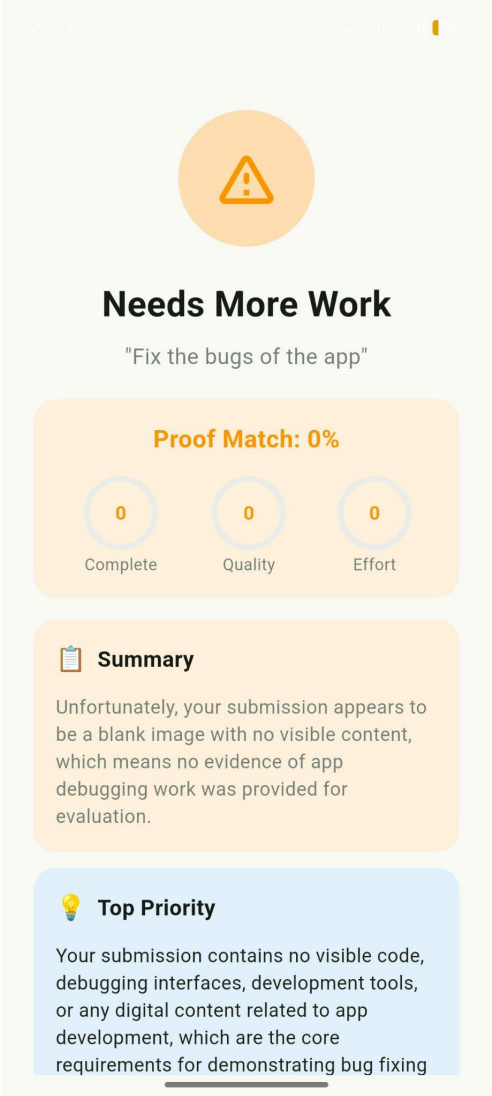

Figure 28: Proof of Work Rejection Page for Sproutly

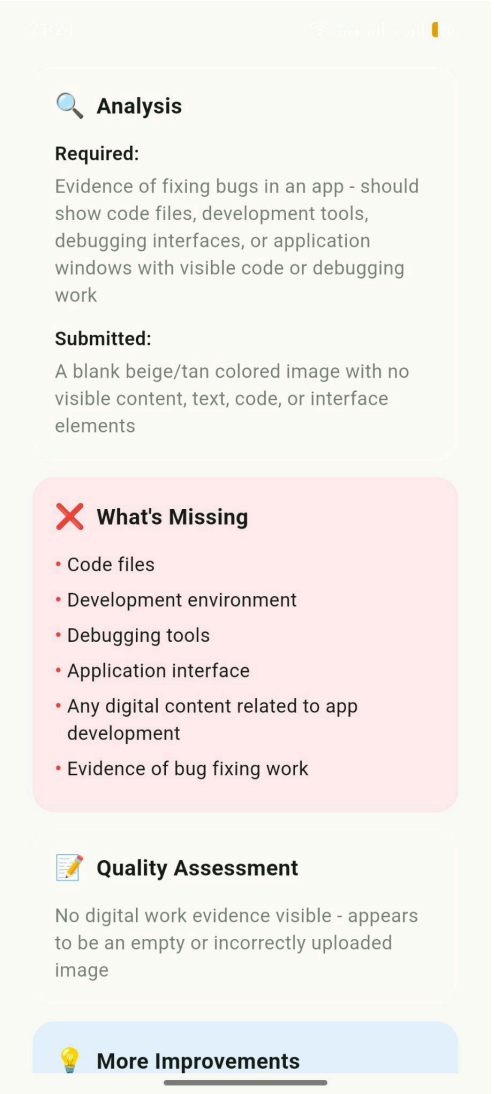

Figure 29: Cont. Proof of Work Rejection Page for Sproutly

After the user submits their proof of work, the AI analyzes whether the proof is adequate and satisfies the requirements of the initial task. AI will give a comprehensive analysis on the work of the user, such as what was missing in the proof or what can be done to improve the work.

9.27 Proof of Work Rejection Page (Cont.)

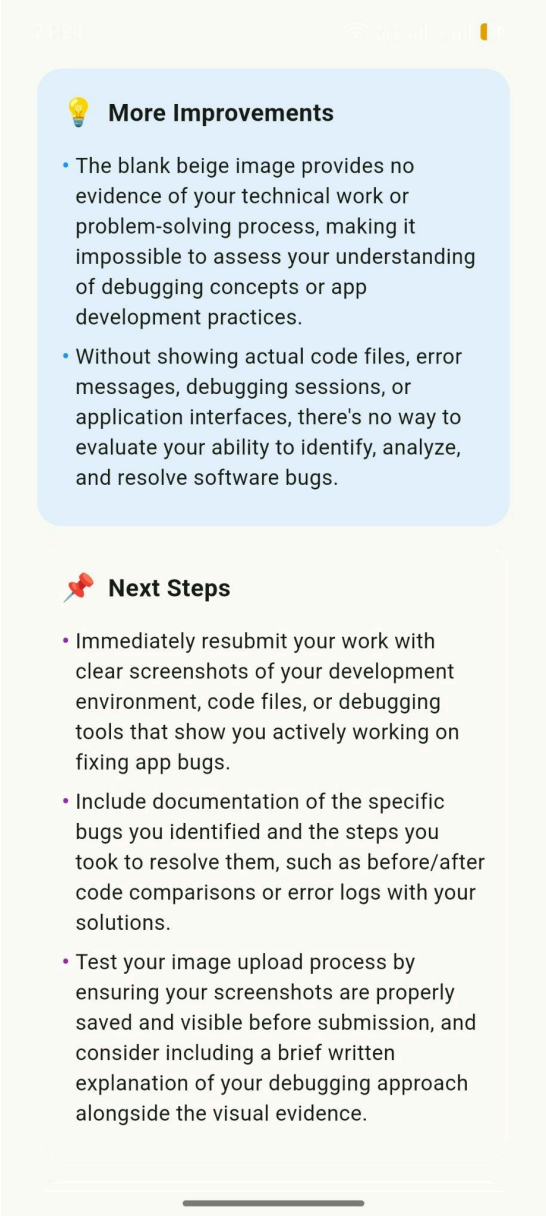

Figure 30: Cont. Proof of Work Rejection Page for Sproutly

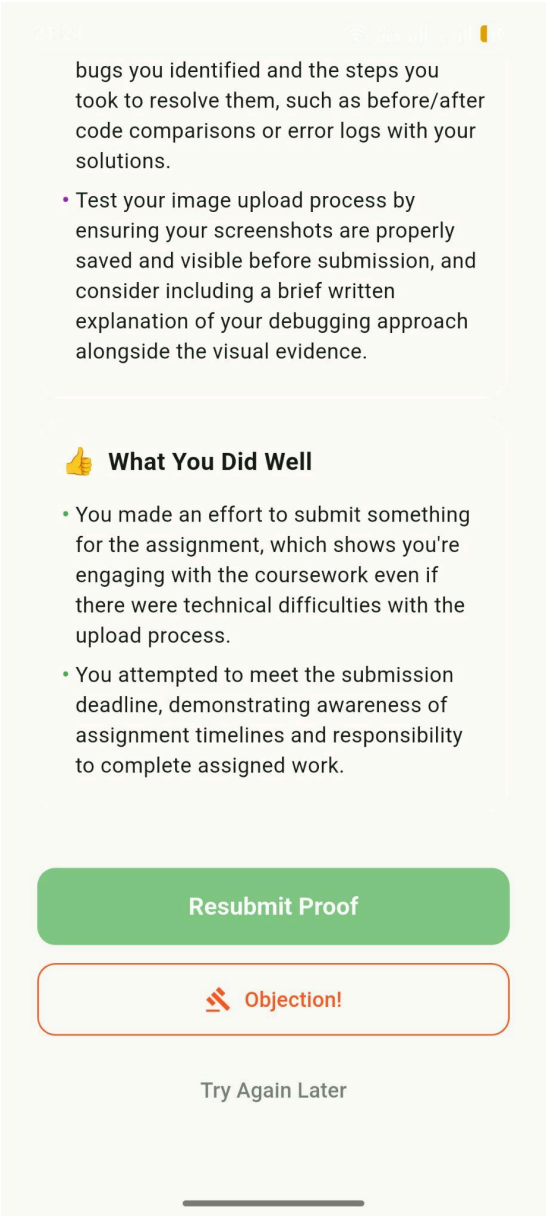

Figure 31: Cont. Proof of Work Rejection Page for Sproutly

When the proof gets rejected, users can resubmit proof, object to AI's analysis or they can choose to try again later which will require them to start a focus session all over again.

9.28 Proof of Work Rejection Page (Cont.)

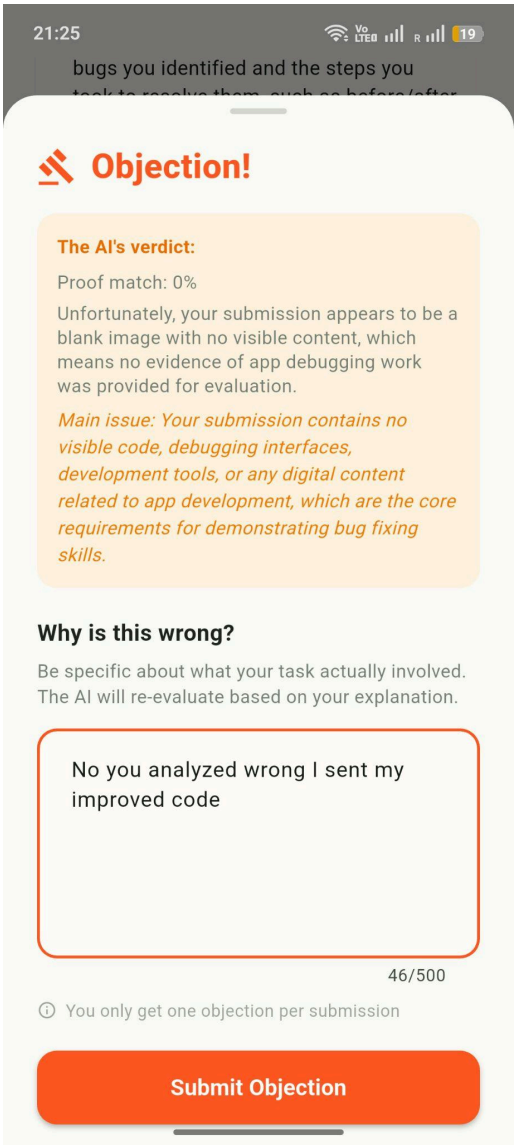

Figure 32: Cont. Proof of Work Rejection Rejection Page for Sproutly

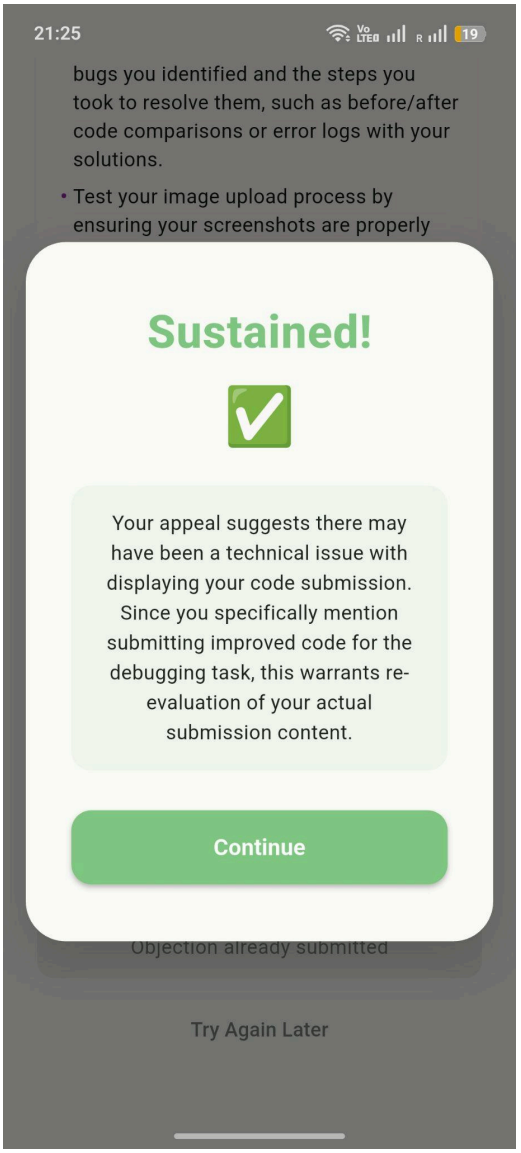

Figure 33: Cont. Proof of Work Rejection Page for Sproutly

In the objection screen, the user can only write a small message for AI to reassess its submission. The user can sustain the objection or deny it. When the coaching mode reaches extreme, it gets harder to get a sustain on objection as well.

9.29 App Blocking Page

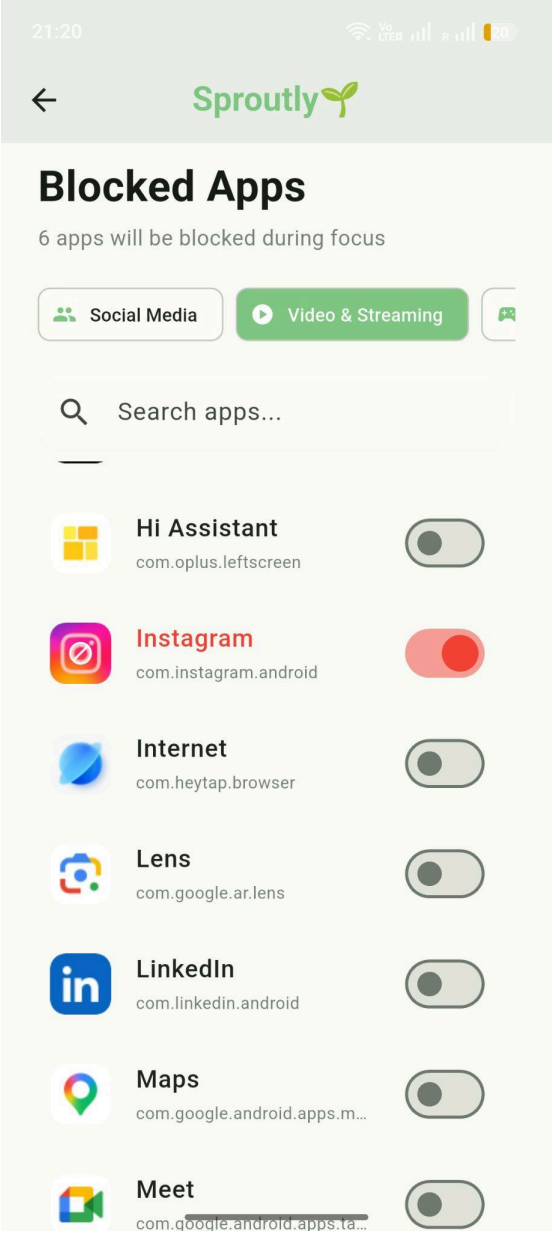

Figure 34: Blocked Apps Preference Page for Sproutly

9.30 Blocked App Overlay

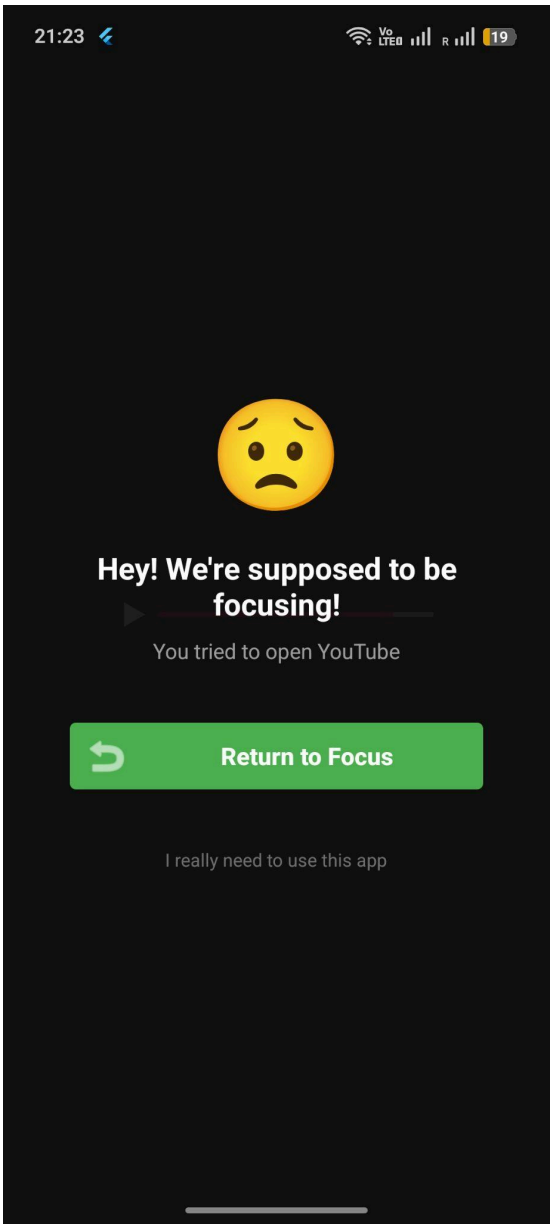

Figure 35: Opened Blocked App During Focus Session Overlay for Sproutly

Except the casual mode, the user can focus while blocking some apps to avoid distractions. The app blocking settings page is accessed through the settings page and all of the apps in the users' phone are visible in it. Users can select the distracting apps and when they try to enter it during a focus session, an overlay will pop up telling them to go back to studying. However, if the user really needs to use this app, then they can override this with the "I really need to use this app" button. After some seconds (depending on the coaching mode), the user will be able to use the app while facing some consequences like XP loss.

9.31 Notes Page

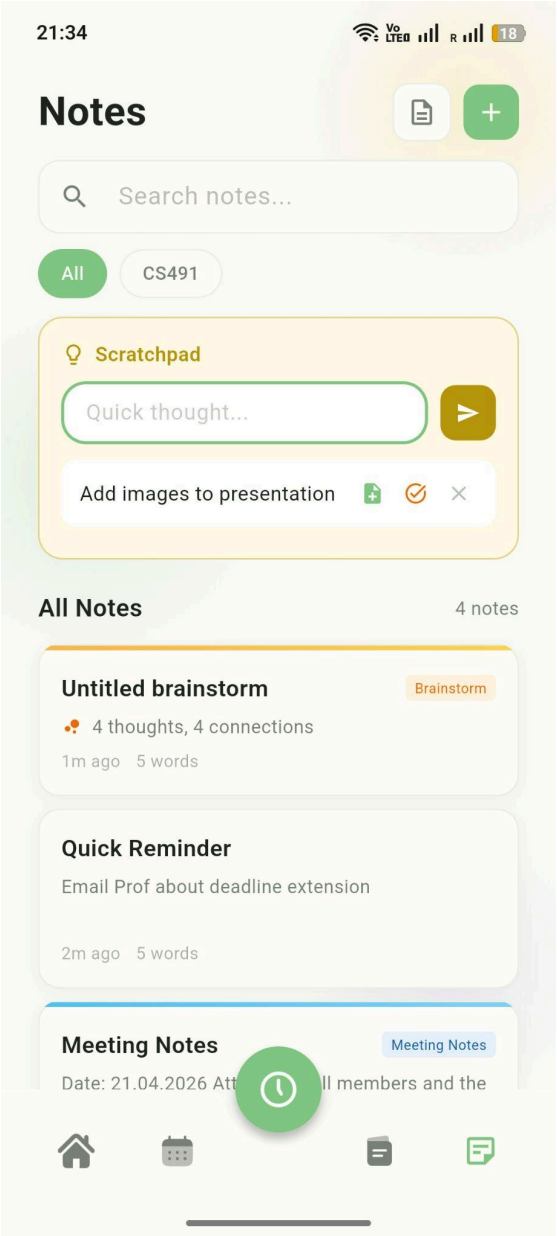

Figure 36: Notes Page for Sproutly

9.32 AI Note Connection Page

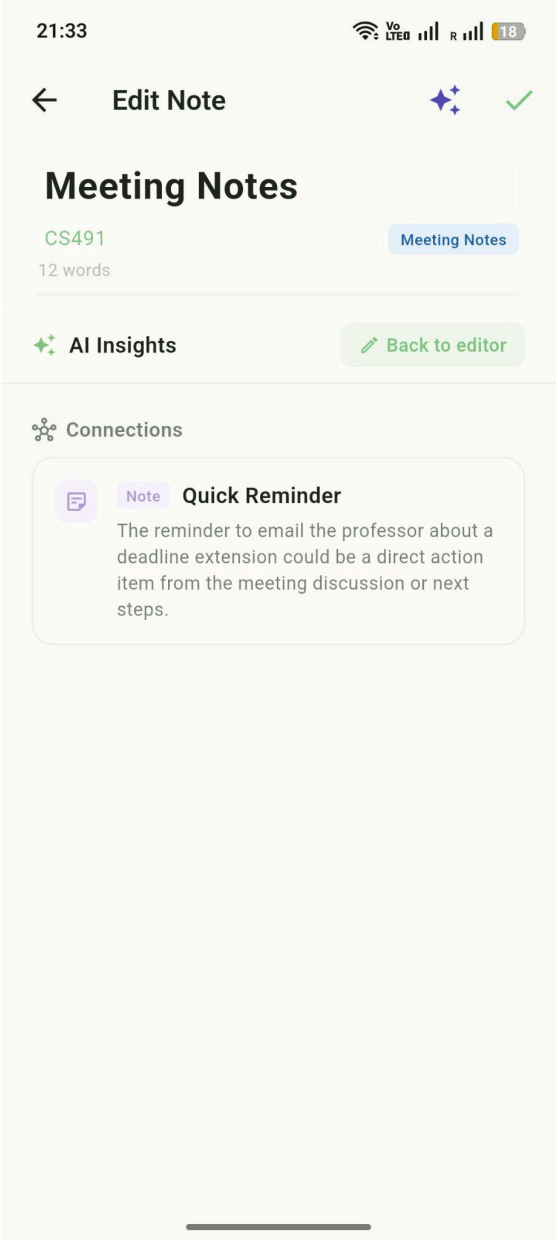

Figure 37: AI Note Connection Page for Sproutly

The notes page shows all saved notes with a search bar at the top and topic filters to narrow down notes by subject. The scratchpad at the top lets the user jot down quick thoughts, which can then be turned into notes or tasks using the icons below it. Notes are listed below the scratchpad, with pinned notes shown first. Each note card shows the title, a preview of the content, the word count, how long ago it was edited, and its template type if it has one.

By tapping the AI icon which looks like three stars inside a note, the user can switch to the AI Insights view. The connections section shows which tasks from the user's task list and notes are related to the note. Each connection includes the task name and a short explanation of why it is relevant. The user can tap "Back to editor" to return to editing the note.

9.33 Brainstorm Template Page

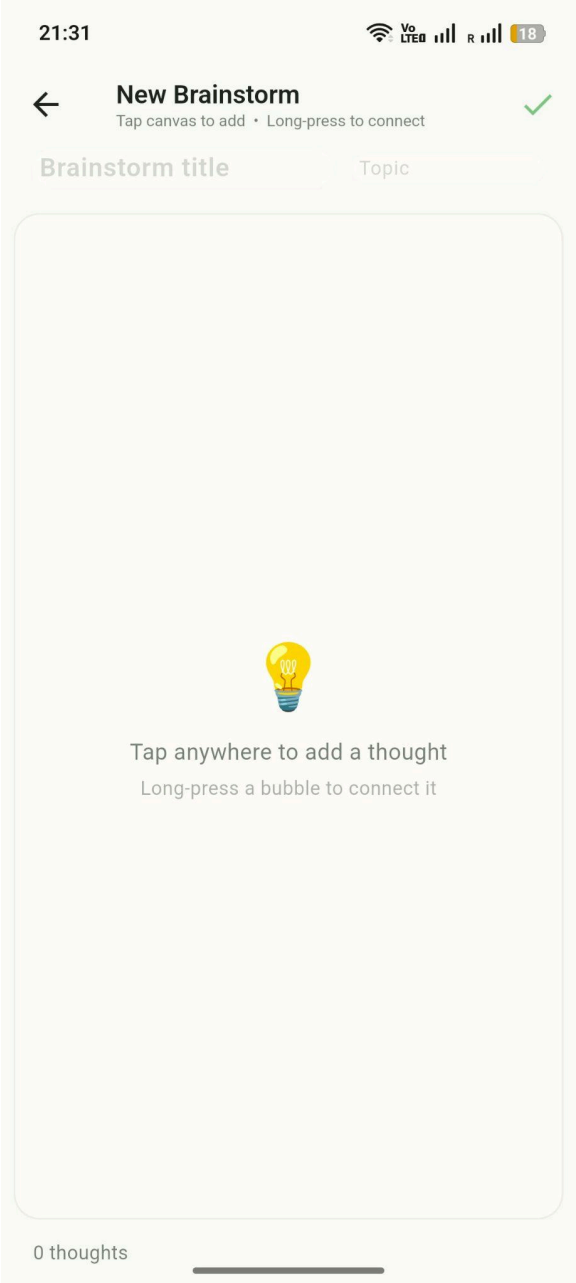

Figure 38: New Brainstorm Page for Sproutly

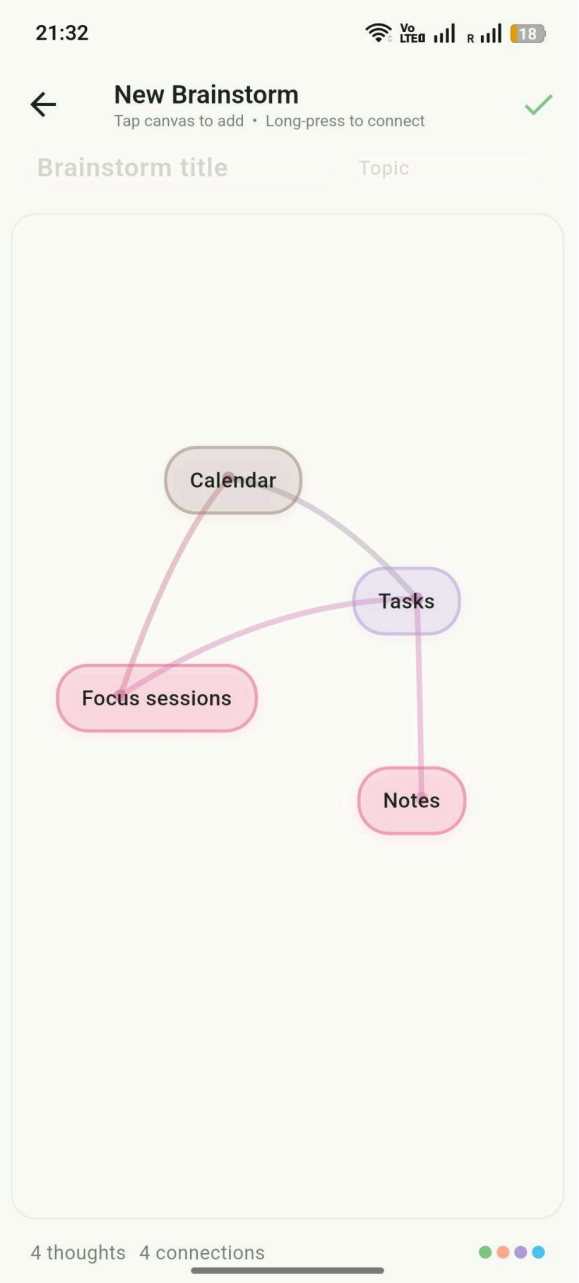

Figure 39: Cont. Brainstorm Page for Sproutly

One of the note templates is the brainstorm template. The user can click on the brainstorm canvas and write their thoughts on the pop up. This note will turn into a bubble. They can add as many bubbles as they want. To connect the bubbles the user needs to click on the bubbles for a long duration and then click on the other bubble they would like to connect. To disconnect bubbles the same procedure has to be done.

9.34 Note Editing Page

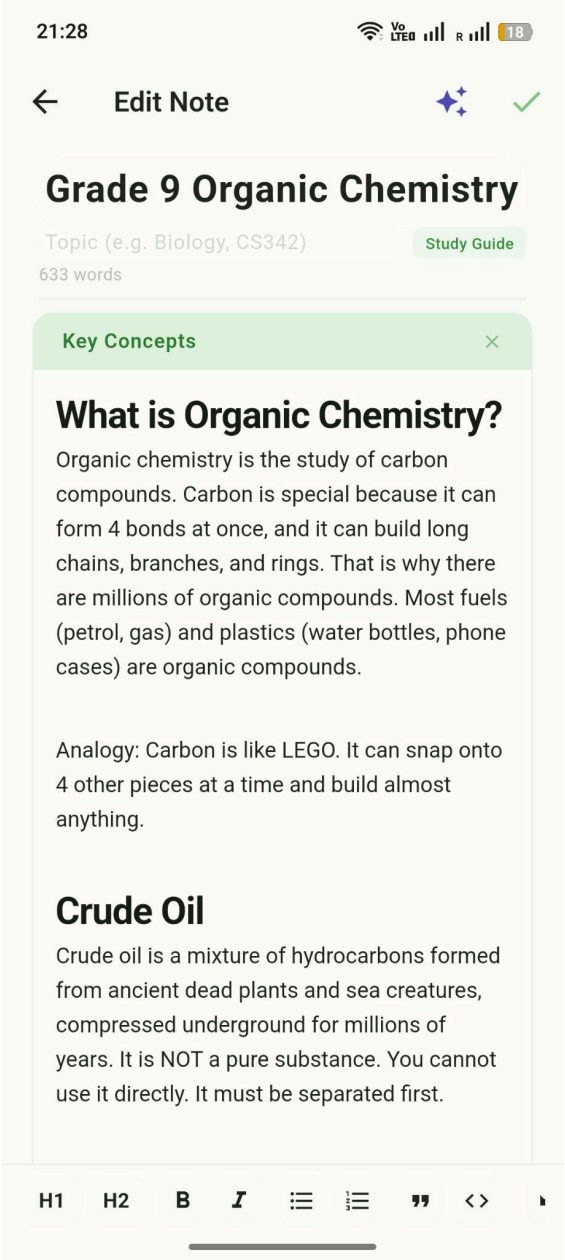

Figure 40: Note Editing Page for Sproutly

9.35 AI Note Summary Page

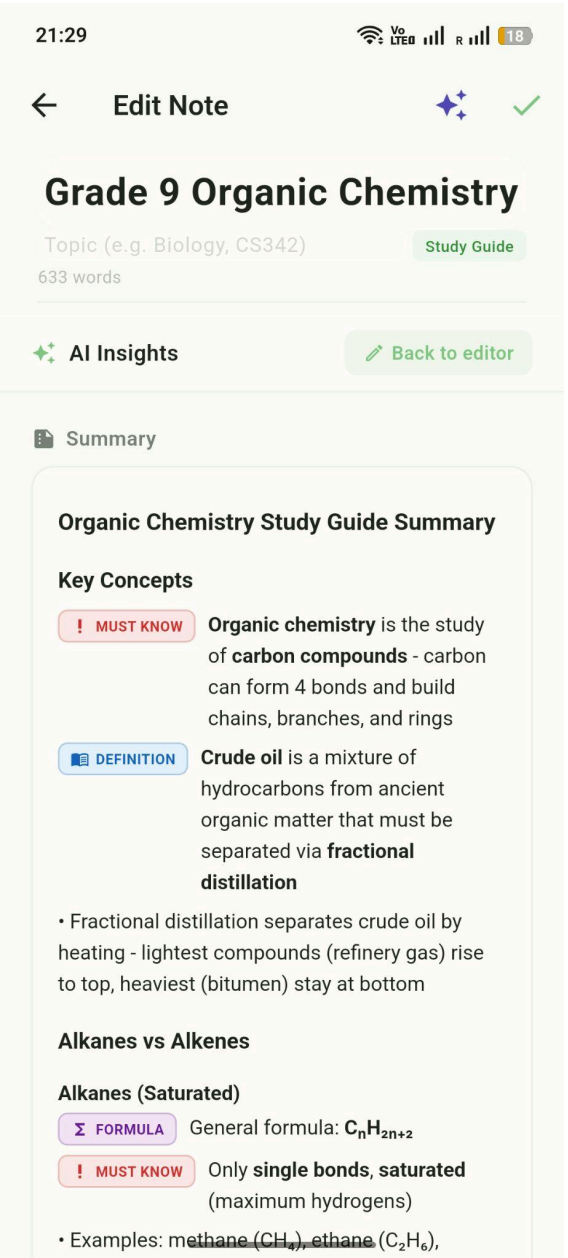

Figure 41: Cont. Note Editing Page for Sproutly

The users can add their own notes to the app either as a template or a free form. The toolbar at the bottom gives them the flexibility to write their notes in many ways. They can add sections and change the colors to their sections by tapping long on the header of them. The users can also benefit from AI, such that they can make AI summarize their notes for them. This functionality is reached from the star button at the top. A slider will come up with options AI summarize and AI connections.

9.36 Note Template Page

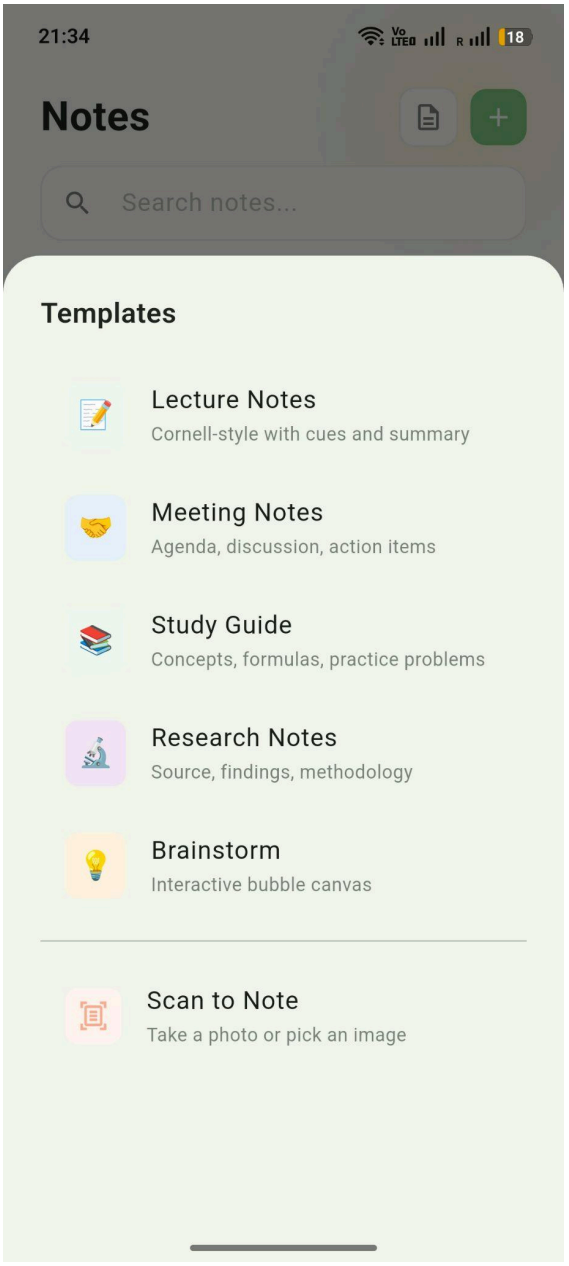

Figure 42: Note Template Page for Sproutly

9.37 Settings Page

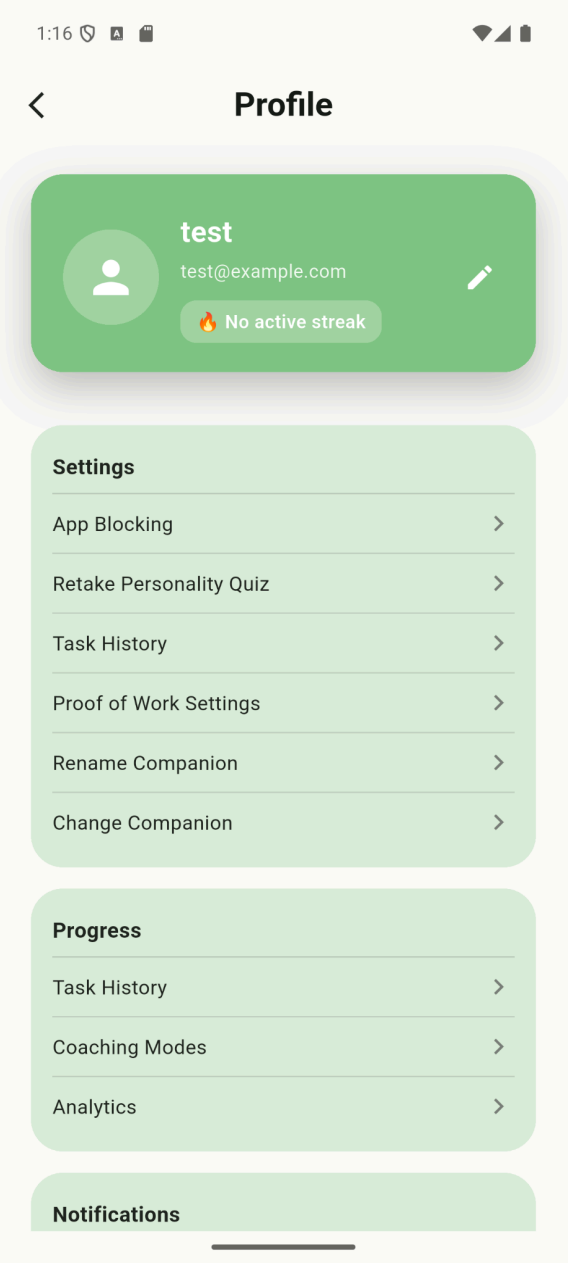

Figure 43: Settings Page for Sproutly

The user can choose their preferred template to create any note.

By clicking the gear icon on the top right of the Home Page (9.9), the user can navigate to the settings page. From here, the user can change their preferences, companions, etc.

9.38 Task History Page

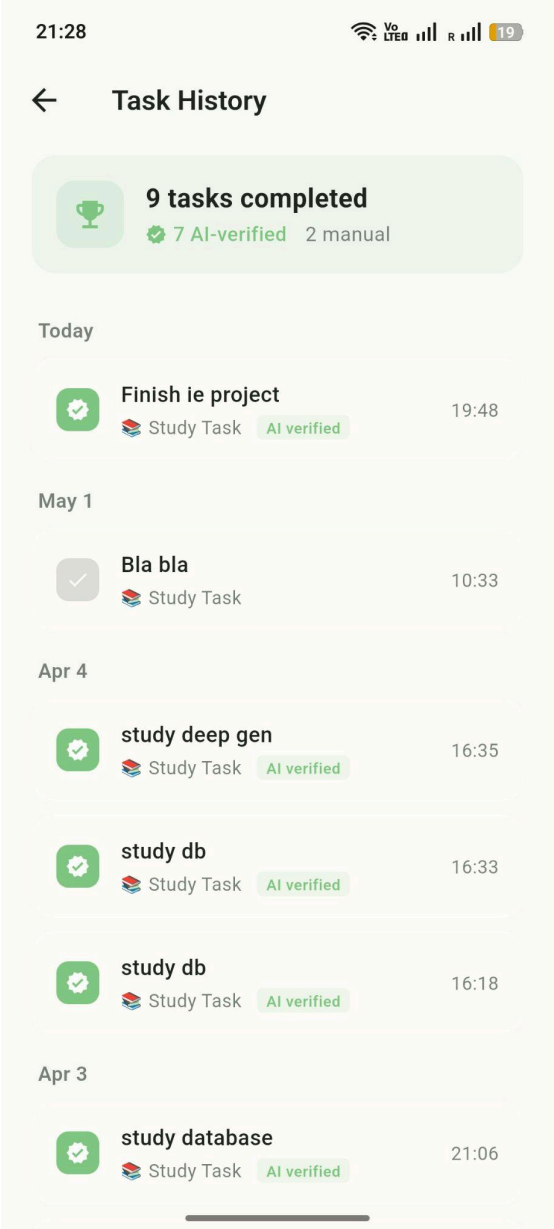

Figure 44: Task History Page for Sproutly

9.39 Achievements Page

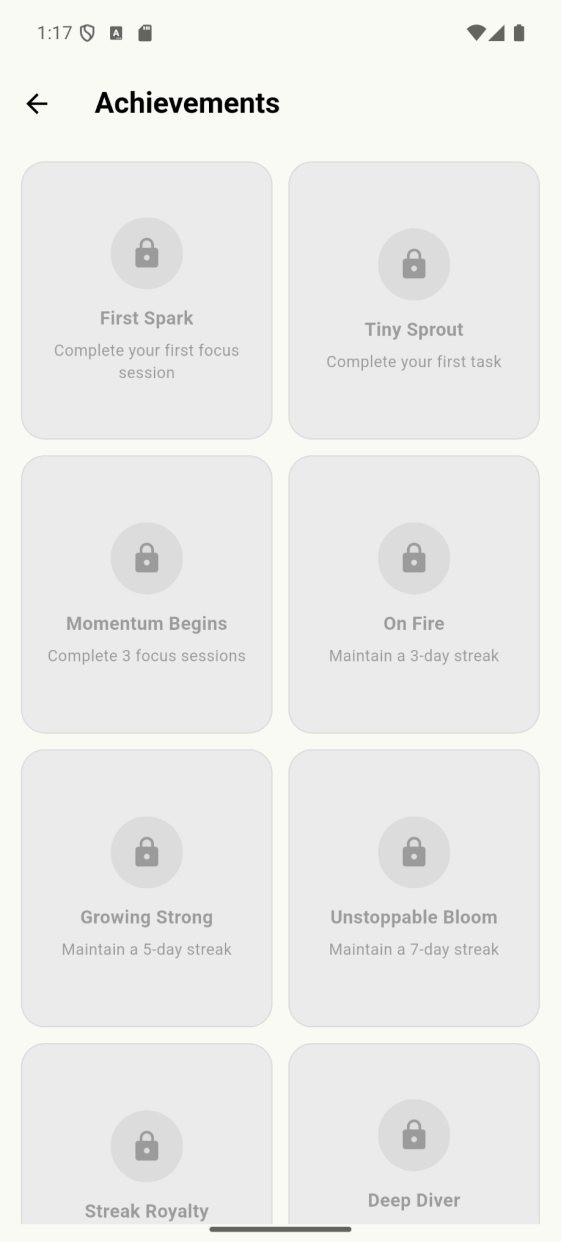

Figure 45: Achievements Page for Sproutly

The user can see the tasks they completed from the Task History page.

By clicking the Achievements card on the Home Page (9.9), the user can navigate to the achievements page and view their achievements. Achievements can be earned through accomplishing various goals, maintaining streaks, completing a certain number of tasks or focus sessions.

9.40 Weekly Review Page

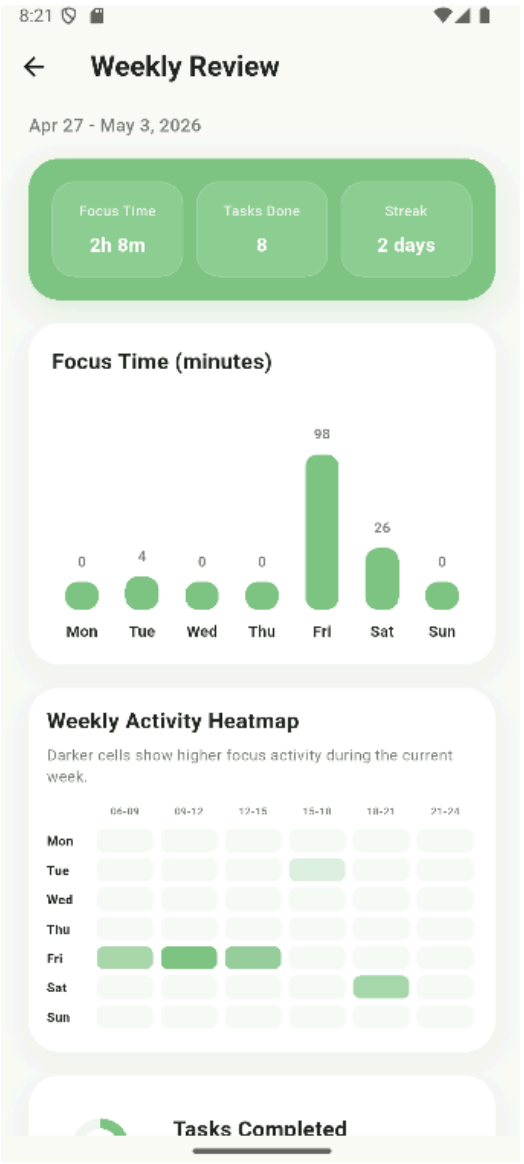

Figure 46: Weekly Review Page for Sproutly

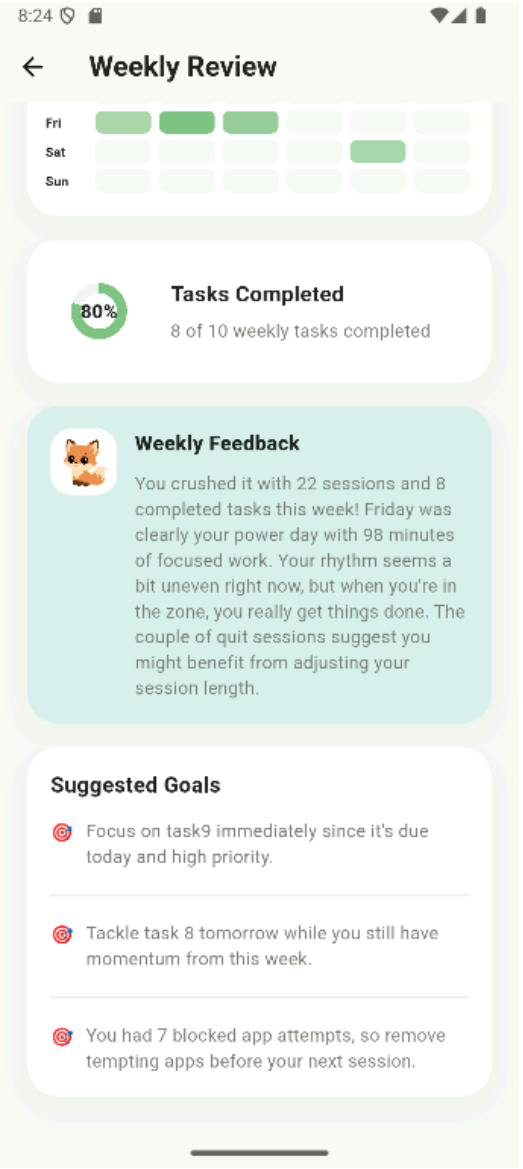

Figure 47: Cont. Weekly Review Page for Sproutly

The user can get an overview of what they did in the past week through the Weekly Review page. This page shows the focus time, number of tasks done in that week and the user's streak. This page also shows a weekly activity heatmap and the task completion percentage for that week, as well as an AI based companion feedback and three different suggestions.

9.41 Mood check-in Page

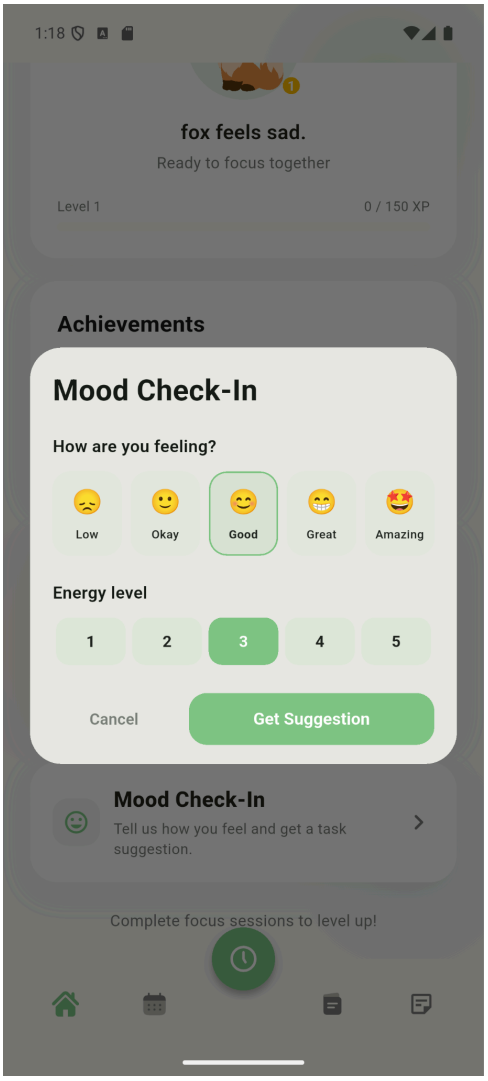

Figure 48: Mood Check-in Page for Sproutly

9.42 Analytics Page

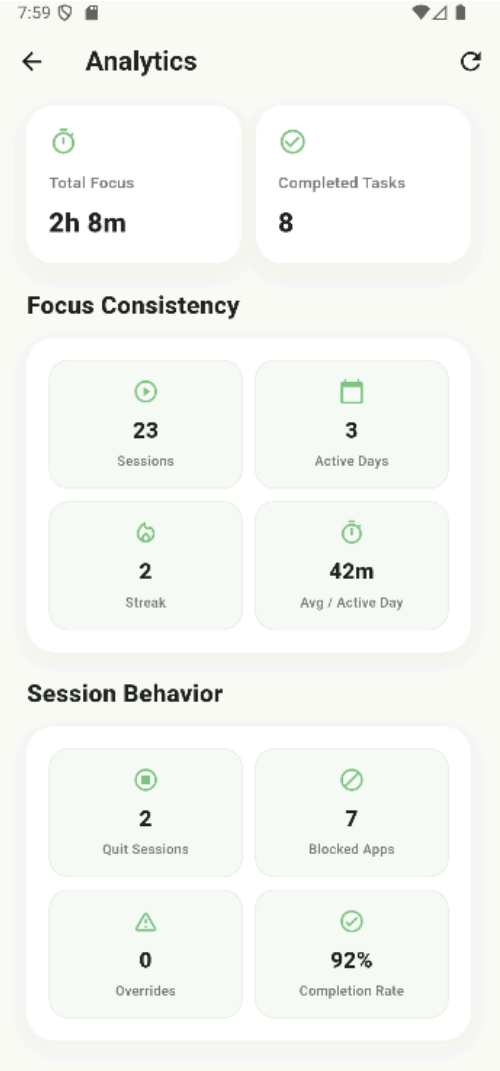

Figure 49: Analytics Page for Sproutly

The user can get a task suggestion by entering how they're feeling and their energy level from the mood check-in page.

The user can see session and focus data such as all sessions completed or number of sessions quit or number of times that the user tried to open a blocked app or override attempts from the analytics page.

10. Glossary

Sproutly: Name of the application in development. May be generically referred to as ‘app’ or ‘application’ throughout the document.
Focus Session: A certain duration during which a user actively focuses and does tasks.
Restriction Mode: The mode that the user chooses, depending on how intense they want the focus restrictions to be, to ensure the user stays focused.
Companion: The character designed to accompany, help, and motivate the user. Companions have animations and customizations.
Streak: The number of times in a row, in days, that the user has utilized the application.
Distraction Inbox: A section where users can write down distracting thoughts during a focus session so they can stay focused.
Coaching Mode: A selection of different modes with different intensity levels and focus features that users can choose based on their preferences.
Personality Test: A test that users answer at the beginning of their registration to set the user’s initial preferences in the app.
OS: Operating System.
UI/UX: User Interface / User Experience. Refers to elements of the app which the user interacts with, and how the app responds to user input.
API: Application Programming Interface. A software tool that allows the app to communicate with other third party applications not directly integrated into the app.
LLM: Large Language Model. A type of general purpose artificial intelligence good at interpreting natural language and certain other human media.
AI: Artificial Intelligence.
IEEE: Institute of Electrical and Electronics Engineers. An institute responsible for setting standards for numerous design, architecture and implementation level concepts for software engineering and other related engineering fields.
OCR: Optical Character Recognition.
KVKK: Kişisel Verilerin Korunması Kanunu. Turkey’s personal data protection law.
HIPAA: Health Insurance Portability and Accountability Act. A US law that protects health related information.
GDPR: General Data Protection Regulation. A data privacy regulation of the European Union.
CORS: Cross-Origin Resource Sharing. A browser security tool controlling which domains can call the API.
UUID: Universally Unique Identifier. A randomly generated string used to tag and trace individual requests through the system logs.

11. References

[1] Google, "Flutter: Build apps for any screen," 2024. [Online]. Available: https://flutter.dev

[2] Google, "Dart programming language," 2024. [Online]. Available: https://dart.dev

[3] Google, "Firebase Authentication," 2024. [Online]. Available: https://firebase.google.com/docs/auth

[4] Rive, Inc., "Rive — Build interactive animations," 2024. [Online]. Available: https://rive.app

[5] S. Ramírez, "FastAPI: Modern, fast web framework for building APIs with Python," 2024. [Online]. Available: https://fastapi.tiangolo.com

[6] Anthropic, "Claude API Documentation," 2025. [Online]. Available: https://docs.anthropic.com

[7] F. Cirillo, The Pomodoro Technique: The Acclaimed Time-Management System That Has Transformed How We Work. New York, NY, USA: Currency, 2018.

[8] SQLite Consortium, "SQLite," 2024. [Online]. Available: https://www.sqlite.org

[9] tekartik, "sqflite: SQLite plugin for Flutter," 2024. [Online]. Available: https://pub.dev/packages/sqflite

[10] Google, "SharedPreferences plugin for Flutter," 2024. [Online]. Available: https://pub.dev/packages/shared_preferences

[11] A. Razazian, "flutter_quill: Rich text editor for Flutter," 2024. [Online]. Available: https://pub.dev/packages/flutter_quill

[12] S. Collings, "Pydantic: Data validation using Python type annotations," 2024. [Online]. Available: https://docs.pydantic.dev

[13] J. Clark, A. Murray, et al., "Pillow: The friendly PIL fork," 2024. [Online]. Available: https://python-pillow.org

[14] Apple, Inc., "WidgetKit," 2024. [Online]. Available: https://developer.apple.com/documentation/widgetkit

[15] Google, "App Widgets Overview," Android Developers, 2024. [Online]. Available: https://developer.android.com/develop/ui/views/appwidgets/overview

[16] IEEE, "IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications," IEEE, 1998.

[17] Object Management Group, "OMG Unified Modeling Language (OMG UML), Version 2.5.1," 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1

[18] ISO/IEC, "ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)," International Organization for Standardization, 2011.

[19] ISO/IEC/IEEE, "ISO/IEC/IEEE 12207:2017 — Systems and software engineering — Software life cycle processes," International Organization for Standardization, 2017.

[20] G. van Rossum, B. Warsaw, and A. Coghlan, "PEP 8 — Style Guide for Python Code," Python Software Foundation, 2001. [Online]. Available: https://peps.python.org/pep-0008/

[21] Republic of Turkey, "Kişisel Verilerin Korunması Kanunu (KVKK), Law No. 6698," Official Gazette, 2016.

[22] European Parliament and Council, "Regulation (EU) 2016/679 — General Data Protection Regulation (GDPR)," Official Journal of the European Union, 2016.

[23] U.S. Department of Health and Human Services, "Health Insurance Portability and Accountability Act of 1996 (HIPAA)," 1996.

[24] Google, "image_picker plugin for Flutter," 2024. [Online]. Available: https://pub.dev/packages/image_picker

[25] Google, "path_provider plugin for Flutter," 2024. [Online]. Available: https://pub.dev/packages/path_provider

[26] D. Patel, "flutter_svg: SVG rendering widget for Flutter," 2024. [Online]. Available: https://pub.dev/packages/flutter_svg